Smart Manufacturing Quality Intelligence System

Project Planning & System Design Document

- Smart Manufacturing Quality Intelligence System
  - 1. Finalized Project Vision
  - 2. Recommended Manufacturing Domain / Product
  - 3. Why This Domain Fits the Problem Statement
  - 4. Final Feature Set
  - 5. Complete User Journey
  - 6. System Architecture
  - 7. Manufacturing Process Simulation Design
  - 8. Stage Intelligence Architecture
  - 9. ML / DL Architecture
  - 10. Quality Prediction — The Fusion Engine
  - 11. Explainable AI
  - 12. Anomaly Detection
  - 13. What-If Simulation
  - 14. “What Should I Change to Make It Pass?” — Recommendation Engine
  - 15. Dynamic Interactive Frontend
  - 16. Backend / API Architecture
  - 17. Database / Data Model
  - 18. Event-Driven / State-Driven Simulation
  - 19. Dataset Strategy
  - 20. Technology Stack (Final, with justification)
  - 21. Development Workflow
  - 22. Implementation Phases
  - 23. MVP vs Enhanced vs Final Showcase
  - 24. Testing & Evaluation
  - 25. Risks and Limitations
  - 26. Final Deliverables
  - 27. Final Architecture Diagram
  - 28. Final End-to-End Technical Workflow
  - 29. Closing Statement on Claims

Smart Manufacturing Quality Intelligence System

Injection Molding Process Quality Prediction, Simulation, Explanation & Recommendation

Document type: Project planning and technical system-design blueprint **Project class:** College Minor Project (B.Tech CSE — AI/ML) **Status:** Design baseline, pre-implementation **Scope rule:** No implementation code in this document. Architecture, contracts, and decisions only.

---

1. Finalized Project Vision

1.1 One-paragraph vision

A software-based **manufacturing quality intelligence system** in which a virtual production batch travels through a sequence of injection-molding process stages. At each stage a dedicated **stage intelligence module** evaluates only the parameters physically relevant to that stage, emits a stage health score, a risk label, and an anomaly flag, and passes an enriched process state forward. A final **fusion quality engine** consumes the raw process parameters, the stage-level intelligence outputs, engineered interaction features, and machine/product context to predict whether the produced part will pass quality inspection. The system then explains the prediction through feature attribution, lets the user mutate parameters in an interactive **what-if simulation**, and runs a **constrained recommendation search** that answers the question *“what should I change to make it pass?”* — always inside documented, physically admissible parameter windows.

1.2 What this project is

- A **decision-support** system for process engineers and operators.
- A **digital workflow simulation** of a manufacturing line, driven by data, rules, and ML inference.
- A **quality prediction** system with calibrated probability and explicit risk semantics.
- A **counterfactual recommendation** system operating under hard manufacturing constraints.
- A professionally engineered full-stack application (API + database + frontend + model store).

1.3 What this project is explicitly NOT

Not this	Reason
A physics engine (CFD / FEA / mold-flow solver)	Out of scope, out of skill budget, and unnecessary for the learning objective
A digital twin in the industrial sense	A true digital twin requires live sensor telemetry bound to a specific physical asset
A validated process-optimization product	No physical trial runs exist to validate any recommendation
A generic “manufacturing platform”	Genericity destroys domain credibility; one process is modelled properly
A one-agent-per-parameter pipeline	Parameters interact; per-parameter agents model interactions incorrectly

1.4 The four claim categories (used consistently throughout)

This document, the software UI, and the final report must keep these four separate. Mixing them is the single largest academic-integrity risk in this project.

```
+-----------------+-------------------------------------------------------------+
| PREDICTS        | A statistical model estimates P(fail) from process parameters |
|                 | learned from data. Claim strength: statistical association.   |
+-----------------+-------------------------------------------------------------+
| SIMULATES       | A software state machine advances a virtual batch through      |
|                 | stages and computes stage indicators from rules + models.      |
|                 | Claim strength: workflow-level, NOT material-physics-level.    |
+-----------------+-------------------------------------------------------------+
| RECOMMENDS      | A constrained search over admissible parameter windows finds   |
|                 | a configuration with lower model-predicted failure probability.|
|                 | Claim strength: model-relative, not plant-validated.           |
+-----------------+-------------------------------------------------------------+
| CANNOT CLAIM    | Any assertion that a recommendation will improve real scrap    |
|                 | rate, real part strength, cycle economics, or tool life,       |
|                 | without physical trial runs and metrology.                     |
+-----------------+-------------------------------------------------------------+
```

Every recommendation surface in the UI carries a model-relative disclaimer. Every explainability panel carries an “association, not causation” note. This is a feature of the project, not a weakness — examiners reward calibrated honesty.

---

2. Recommended Manufacturing Domain / Product

2.1 Candidate evaluation

Five realistic candidate domains were evaluated against the parameters named in the problem statement (Temperature, Pressure, Machine Speed, Production Time, Raw Material Measurements, Machine ID).

Criterion	Injection Molding	CNC Machining	Semiconductor Fab (SECOM-like)	Steel / Metal Rolling	Food & Beverage Filling
Temperature is a first-class parameter	Yes (barrel, nozzle, mold)	Weak (tool/coolant only)	Yes but opaque	Yes	Weak
Pressure is a first-class parameter	Yes (injection, packing, back pressure)	No	No	Yes (roll force)	Yes (fill pressure)
Speed is a first-class parameter	Yes (injection speed, screw RPM)	Yes (feed/spindle)	No	Yes	Yes
Time is a first-class parameter	Yes (cycle, cooling, hold time)	Yes	Yes	Weak	Yes
Raw-material measurements matter	Yes (moisture, MFI, regrind %)	Weak	No	Yes	Weak
Machine ID / multi-machine context	Yes (cavity + machine variance)	Yes	Yes	Yes	Yes
Natural PASS/FAIL inspection label	Yes (short shot, flash, warp, sink)	Partly (tolerance)	Yes but binary and opaque	Yes	Weak
Clear multi-stage decomposition	**Excellent**	Poor (single continuous cut)	Excellent but 590 anonymous sensors	Moderate	Moderate
Public dataset availability	Moderate	Moderate	Good (UCI SECOM)	Moderate	Poor
Dataset interpretability	**Good** (named physical columns)	Good	**Very poor** (anonymised signals)	Moderate	Poor
Visualizable as a stage pipeline	**Excellent**	Poor	Poor	Moderate	Good
What-if analysis is physically meaningful	**Excellent**	Moderate	Meaningless (anonymous features)	Moderate	Weak
Domain literature for constraint windows	**Abundant**	Abundant	Scarce/NDA	Abundant	Moderate
Feasible without physics engine	Yes	Yes	Yes	Yes	Yes
Student-team feasibility	**High**	High	Low (feature-engineering hell)	Moderate	Moderate

2.2 Decision

> ***FINAL DOMAIN: Thermoplastic injection molding of a small technical component — specifically a polypropylene (PP) automotive interior clip / electrical enclosure housing produced on a hydraulic/electric injection molding machine,***

> ***multi-machine shop floor, single-cavity or 4-cavity tooling.***

Concrete product chosen: a 30–60 g PP electrical junction-box housing. It is small (short cycle, plausible high-volume dataset), geometrically simple enough to reason about, and its dominant defect modes map directly onto the six parameters in the problem statement.

2.3 Why the other candidates were rejected

- **Semiconductor fab (SECOM):** the dataset is 590 anonymised sensor channels. Nothing can be explained, no what-if is meaningful, no recommendation is defensible. It defeats the entire purpose of the project.
- **CNC machining:** pressure is absent, and quality is a continuous dimensional-tolerance problem, not naturally a staged pipeline. The stage architecture would be artificial.
- **Rolling / metal forming:** physically rich but the quality mechanisms (metallurgical) are far harder to defend at student level, and constraint windows are alloy-specific.
- **Filling lines:** too few interacting parameters; what-if analysis would be trivial.

---

3. Why This Domain Fits the Problem Statement

3.1 Parameter-to-domain mapping

PS parameter	Injection molding realisation	Stage where it dominates
Temperature	Barrel zone temps, nozzle temp, melt temp, mold/tool temp	Thermal (plasticizing), Cooling
Pressure	Injection pressure, packing/holding pressure, back pressure, clamp force	Injection, Packing
Machine Speed	Injection velocity (mm/s), screw rotation speed (RPM)	Injection, Thermal
Production Time	Cycle time, injection time, hold time, cooling time	All stages
Raw Material Measurements	Resin moisture %, melt flow index (MFI), regrind fraction, batch lot, colourant %	Material
Machine ID	Machine/tool identity, cavity ID, machine age, last-maintenance gap	Context (all stages)

Every parameter in the original problem statement has a real, named, documented physical role. Nothing is invented to fit.

3.2 Defect modes give the PASS/FAIL label physical meaning

Defect	Dominant causes	Demonstrates
Short shot (incomplete fill)	Melt temp too low, injection pressure/velocity too low, cycle too fast	Temp x Pressure interaction
Flash (material escapes parting line)	Pressure too high, clamp too low, melt too hot	Non-monotonic pressure effect
Sink marks / voids	Insufficient packing pressure or hold time, thick section cooling	Pressure x Time interaction
Warpage	Uneven/insufficient cooling, mold temp imbalance, short cooling time	Cooling-stage intelligence
Splay / silver streaks	Resin moisture above spec, excessive melt temp, shear degradation	Material-stage intelligence
Burn marks / degradation	Melt residence time too long, temp too high, injection speed too high	Thermal anomaly detection

This table is the **conceptual backbone of the project**. It justifies the stage decomposition, it justifies the interaction features, it justifies the constraint windows, and it gives the recommendation engine its safety rails.

3.3 Why the parameters cannot be treated independently

Key documented couplings, which the architecture must respect:

1. **Melt temperature x injection pressure** — higher melt viscosity (cold melt) requires higher pressure for the same fill. Judging pressure without temperature is meaningless.
2. **Injection velocity x melt temperature** — shear heating means high velocity raises effective melt temperature; the pair can jointly cross a degradation threshold that neither crosses alone.
3. **Packing pressure x hold time x part thickness** — sink/void risk depends on the product of pressure and time until gate freeze, not either alone.
4. **Mold temperature x cooling time** — a hot mold with a short cooling time ejects a soft part; the same cooling time with a cold mold is fine.
5. **Resin moisture x melt temperature** — moisture only becomes splay at elevated temperatures.

This is precisely why a per-parameter agent architecture is rejected and a per-stage intelligence architecture is adopted. A “Temperature Agent” would have to independently decide whether 250 °C is safe — a question that has no answer without knowing velocity, resin MFI, and moisture.

---

4. Final Feature Set

4.1 Feature inventory with priority

Priority key: **P0** = MVP-blocking, **P1** = enhanced, **P2** = showcase, **P3** = explicitly excluded.

#	Feature	Priority	Notes
F1	Batch / machine / product creation and selection	P0	Minimal entity set
F2	Parameter entry with validated admissible ranges	P0	Pydantic-enforced
F3	Stage-by-stage simulation state machine	P0	Core differentiator
F4	Six stage intelligence modules	P0	Core differentiator
F5	Fusion quality model (PASS/FAIL + probability)	P0	Core ML deliverable
F6	Stage-level risk + anomaly indicators	P0	Feeds the UI and fusion
F7	Global + local explainability (SHAP)	P0	Required by PS
F8	Animated process-flow visualization	P0	The “wow” surface
F9	Live event log / timeline	P0	Cheap, high perceived value
F10	What-if parameter mutation + delta panel	P0	Required by PS
F11	Constrained recommendation engine	P0	The headline feature
F12	Scenario comparison (A vs B)	P1	Natural extension of F10
F13	Simulation history + analytics dashboard	P1	DB-backed
F14	Parameter sensitivity curves (1-D sweeps)	P1	Very cheap, very convincing
F15	Dedicated unsupervised anomaly detector	P1	Isolation Forest on process vector
F16	Model registry / version metadata surface	P1	MLflow-lite
F17	Pause / resume / step-through / speed control	P1	Demo control
F18	Deep-learning fusion variant (MLP / TabNet-lite)	P2	Only as a benchmarked alternative
F19	Batch CSV upload + bulk scoring	P2	Useful for analytics page
F20	Docker Compose deployment	P2	Presentation value
F21	Multi-user auth, roles, RBAC	**P3**	Out of scope. Single-tenant prototype
F22	Real-time PLC/OPC-UA/MQTT ingestion	**P3**	No hardware exists
F23	3D mold/part rendering (Three.js)	**P3**	Cost >> value; 2D SVG is better
F24	Reinforcement-learning process controller	**P3**	Unvalidatable, no environment
F25	Microservices / Kubernetes	**P3**	Modular monolith is correct here
F26	Physics/mold-flow solver	**P3**	Explicitly forbidden by the brief

4.2 The three headline features (what the project is judged on)

1. **Staged intelligence pipeline** — architecture that mirrors the physical process.
2. **What-if + counterfactual recommendation under hard constraints** — goes beyond classification.
3. **Explainable, honestly scoped decision support** — risk semantics separated from model confidence.

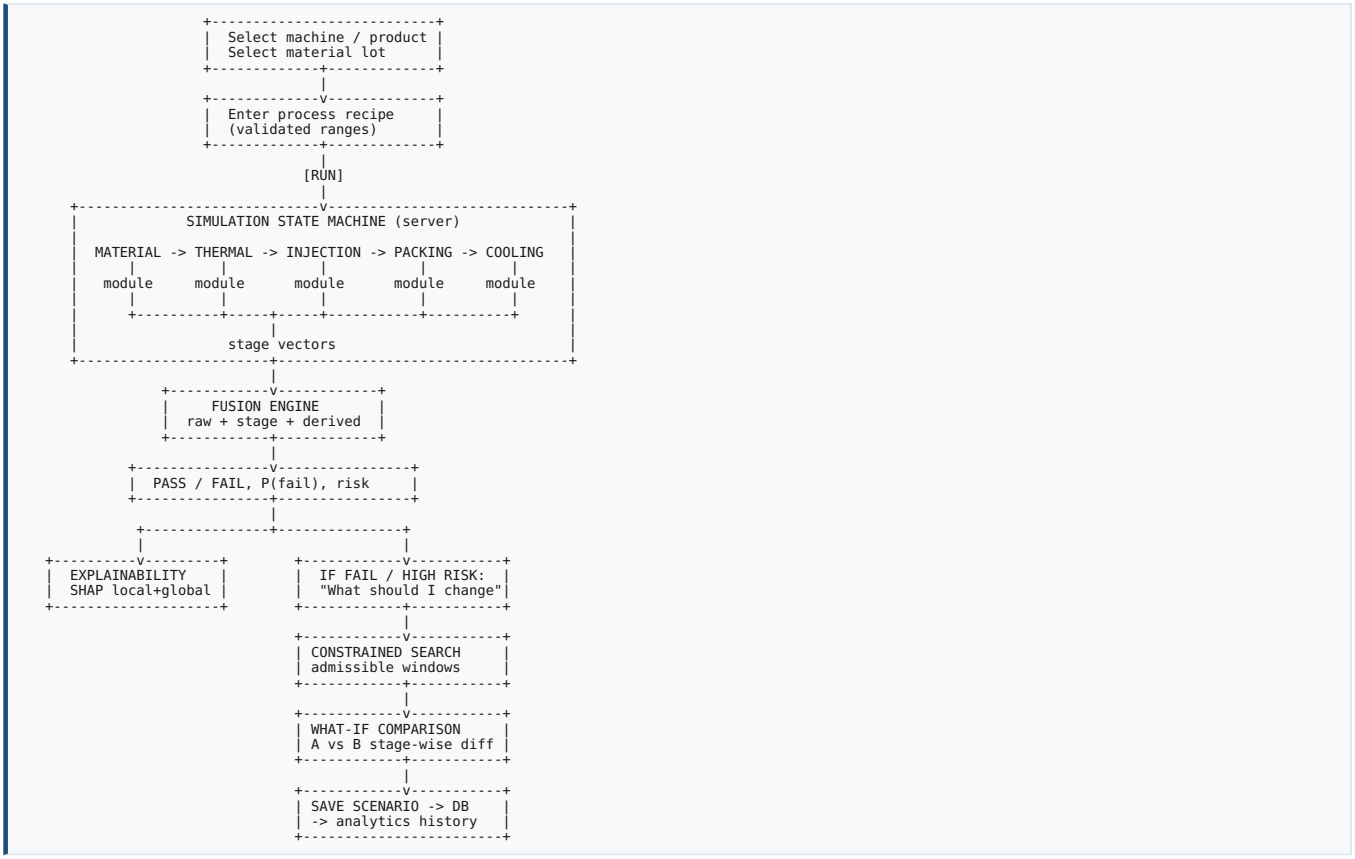
---

5. Complete User Journey

5.1 Narrative walkthrough

```
 1.  User opens the Process Control workspace.
 2.  Selects a machine (e.g. MC-03, 180-ton electric, 4-cavity tool).
 3.  Selects a product/mold (PP junction-box housing, 42 g shot weight).
 4.  Selects or creates a material lot (MFI 12, moisture 0.04%, regrind 15%).
 5.  Enters or loads the process parameter set (setpoint recipe).
 6.  Client-side + server-side validation flags out-of-range entries immediately.
 7.  User presses RUN. A simulation run record is created; run_id returned.
 8.  Stage 1 MATERIAL_CONDITIONING becomes ACTIVE in the UI.
       - Material intelligence module evaluates moisture, MFI, regrind, drying time.
       - Emits health 0.86, risk LOW, anomaly false, notes: "regrind 15% near limit".
 9.  Stage advances to PLASTICIZING / THERMAL.
       - Thermal module evaluates barrel profile, melt temp, screw RPM, back pressure,
         residence time.
       - Emits health 0.61, risk MEDIUM, note: "melt temp 268C with residence 41s ->
         thermal degradation exposure".
10.  Stage advances to INJECTION / FILL.
       - Process module evaluates injection velocity, injection pressure, fill time,
         and the melt-temperature context inherited from the previous stage.
11.  Stage advances to PACKING / HOLDING.
       - Packing module evaluates hold pressure, hold time, pressure-time integral,
         cushion.
12.  Stage advances to COOLING / EJECTION.
       - Cooling module evaluates mold temp, cooling time, part thickness proxy,
         ejection force proxy.
13.  Stage advances to QUALITY_INSPECTION.
       - Fusion engine consumes raw parameters + all five stage vectors + engineered
         interactions + machine context.
14.  FINAL_RESULT: prediction FAIL, P(fail) = 0.71, calibrated; operational risk HIGH.
15.  Explainability panel opens: ranked SHAP attributions, plus the stage that
     contributed the largest risk mass.
16.  User clicks "What should I change?".
17.  Recommendation engine runs a constrained search inside admissible windows,
     respecting locked parameters and per-parameter change costs.
18.  Returns 3 candidate recipes, each with predicted P(fail), the delta from the
     current recipe, the list of changes, and a constraint-compliance stamp.
19.  User selects candidate #2 and sends it to What-If.
20.  What-if re-runs the pipeline; UI shows side-by-side stage-by-stage diff:
     which stage health improved, which risk labels changed, final verdict flip.
21.  User tweaks one value manually (cooling time 27 -> 25 s) and re-runs.
22.  User saves the scenario. It is persisted and appears in history/analytics.
23.  Analytics page shows failure-rate patterns across machines, parameters,
     and stages over all past runs.
```

5.2 Journey as a flow diagram



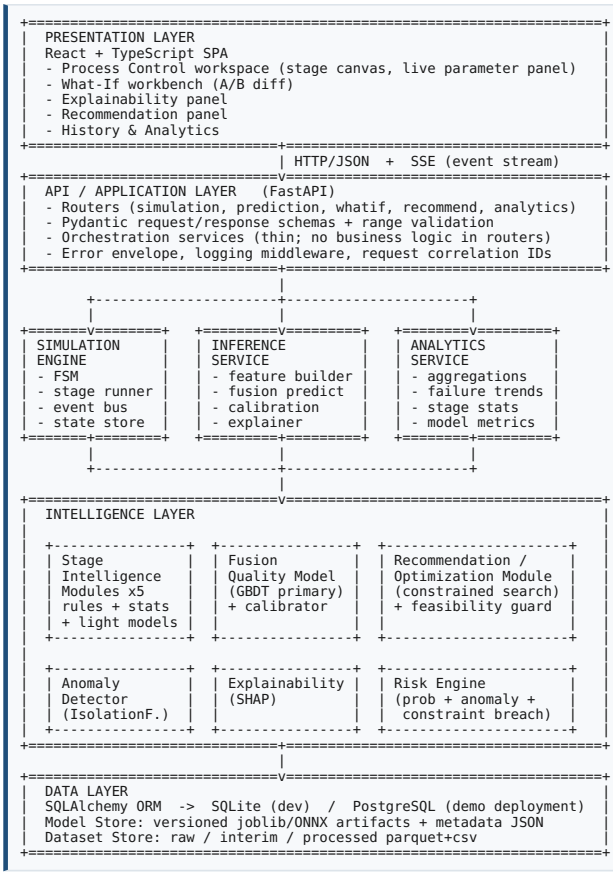
---

6. System Architecture

6.1 Architectural style

Modular monolith (single FastAPI process, strict internal package boundaries) with a separate SPA frontend. Microservices are rejected: they would add deployment complexity and network failure modes without any benefit at this scale.

6.2 Layered view



6.3 Module boundary rules

1. The API layer never imports scikit-learn. It talks to services only.
2. The simulation engine never loads model files directly. It calls the inference service interface.
3. Stage intelligence modules are pure functions of `(stage_inputs, upstream_context)` with no database access — this makes them unit-testable and re-runnable for what-if.
4. The recommendation module may only call the inference service and the constraint registry. It cannot mutate simulation state.
5. The constraint registry (admissible parameter windows) is a **single source of truth** used by: Pydantic validation, the UI sliders, the recommendation search bounds, and the risk engine. It is a versioned YAML/JSON file, not scattered constants.

---

7. Manufacturing Process Simulation Design

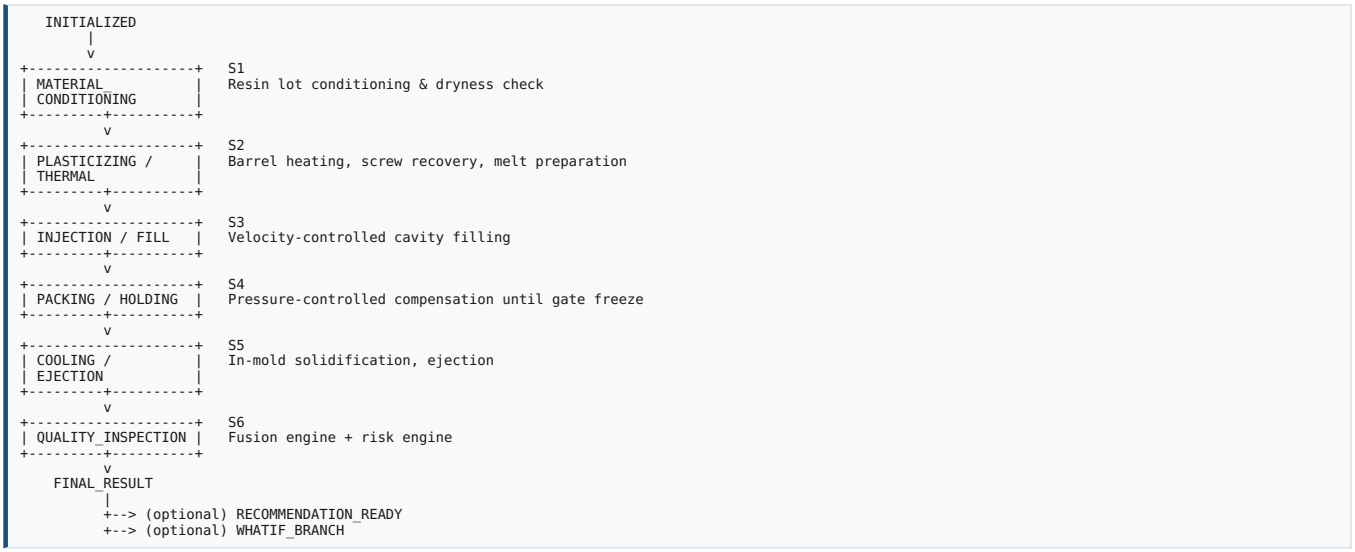
7.1 Physical simulation vs software/process simulation

	Physical simulation	Software / process simulation (this project)
What is computed	Governing equations of matter: momentum, energy, viscoelastic flow, crystallisation, shrinkage fields	Discrete workflow state transitions and data-driven indicators
Method	CFD / FEA / mold-flow solver on a meshed geometry	Finite state machine + rule evaluation + ML inference
Inputs	Mesh, material rheology curves, boundary conditions	Process setpoints, material lot attributes, machine context
Outputs	Pressure fields, temperature fields, weld-line locations, warp displacement	Stage health scores, risk labels, anomaly flags, P(fail)
Validity claim	Approximates physical reality within model error	Approximates the **statistical relationship** observed in the dataset
Compute cost	Minutes to hours per run	Milliseconds per run
Requires	Commercial solver, geometry, material database	Tabular dataset, documented parameter windows

Statement that must appear in the UI, the report, and the viva:

> *"This system performs a software-level process simulation. It advances a virtual batch through a modelled sequence of manufacturing stages and evaluates each stage using rules and machine-learning models fitted to process data. It does **not** solve the physics of polymer melt flow, heat transfer, or shrinkage, and it does not compute material behaviour. Stage indicators are data-driven and rule-driven abstractions, not physical field solutions."*

7.2 The stage sequence (final)



This sequence is the real, textbook injection-molding cycle order. That is the point: the software mirrors the process, so the demo is self-explanatory to anyone who knows molding.

7.3 Formal state machine

States: `INITIALIZED, MATERIAL_CONDITIONING, PLASTICIZING, INJECTION, PACKING, COOLING, QUALITY_INSPECTION, FINAL_RESULT, PAUSED, ABORTED`

Auxiliary per-stage substates: `PENDING -> RUNNING -> COMPLETED | BLOCKED`

Transition table

From	Event	Guard	To	Side effects
INITIALIZED	`RUN`	parameters valid	MATERIAL_CONDITIONING	create run row, emit `run.started`
S_k RUNNING	`STAGE_DONE`	stage result persisted	S_{k+1} PENDING	emit `stage.completed`, `stage.started`
S_k RUNNING	`BLOCK`	critical constraint breach and `halt_on_critical=true`	BLOCKED	emit `stage.blocked`, offer override
any RUNNING	`PAUSE`	—	PAUSED	freeze timer, retain state
PAUSED	`RESUME`	—	previous state	resume timer
any	`RESET`	—	INITIALIZED	clear stage results, keep parameters
any	`ABORT`	—	ABORTED	persist partial run
COOLING	`STAGE_DONE`	all 5 stage vectors present	QUALITY_INSPECTION	invoke fusion engine
QUALITY_INSPECTION	`PREDICTED`	—	FINAL_RESULT	persist prediction + SHAP

Design decision — BLOCKED state. A critical breach (e.g. melt temperature above the resin's documented degradation ceiling) should not silently produce a confident PASS. The engine marks the stage BLOCKED, surfaces the reason, and requires an explicit user override to continue. This is a small feature with large credibility payoff: it demonstrates that the system respects domain constraints over model output.

7.4 Simulation timing model

The simulation is **not** real-time-physical. Stage durations in the UI are **presentation timings**, decoupled from the (millisecond) computation.

- `speed_factor` in {0.5x, 1x, 2x, instant}.
- Default presentation duration per stage: 1200–2000 ms, so a full run takes ~8–10 s — long enough to look alive, short enough for a demo.
- "Instant" mode bypasses animation entirely and is what the what-if and recommendation paths use internally.

Honest labelling: the UI shows "presentation speed", not "process time". The *modelled* process times (cycle time, cooling time, hold time) remain parameters, displayed separately and never confused with animation duration.

7.5 SimulationState object (conceptual schema)

```
SimulationState
+-- run_id                    UUID
+-- status                    enum(state machine)
+-- created_at, updated_at
+-- context
|     +-- machine_id, machine_profile (tonnage, type, age, last_maintenance_days)
|     +-- product_id, mold_id, cavity_count, nominal_shot_weight
|     +-- material_lot (resin_grade, mfi, moisture_pct, regrind_pct, colourant_pct)
+-- parameters                (the recipe: ~14-18 numeric + categorical fields)
+-- stage_results[]
|     +-- stage_name
|     +-- substate
|     +-- inputs_used[]
|     +-- health_score        float 0..1
|     +-- risk_level          enum
|     +-- anomaly             {flag, score, method}
|     +-- indicators{}        stage-specific derived metrics
|     +-- notes[]             human-readable findings
|     +-- constraint_breaches[]
|     +-- duration_ms
+-- aggregate_risk            {level, drivers[]}
+-- prediction
|     +-- label PASS|FAIL
|     +-- p_fail (calibrated)
|     +-- model_version
|     +-- confidence_band
+-- explanation
|     +-- local_attributions[]
|     +-- stage_risk_mass[]
+-- recommendations[]         (populated on demand)
+-- timeline[]                ordered events with timestamps
+-- parent_run_id             (set for what-if branches)
+-- scenario_label            (user-assigned)
```

7.6 Stage execution algorithm (pseudocode-free description)

For each stage, in order:

1. Mark stage RUNNING; emit `stage.started` event.
2. Select the stage's input slice from parameters + context.
3. Merge in **upstream context** — a small dictionary of upstream-derived values the stage physically depends on (e.g. INJECTION receives `effective_melt_temp` and `melt_viscosity_proxy` from THERMAL).
4. Run the stage's deterministic rule checks against the constraint registry.
5. Compute stage indicators (derived quantities).
6. Run the stage's statistical/ML component if present.
7. Combine into `health_score` and `risk_level` via the stage's documented aggregation.
8. Run anomaly scoring for the stage's feature slice.
9. Persist the stage result; emit `stage.completed` with the payload.
10. Advance.

Because steps 2–8 are pure, the entire stage chain is **replayable** and **cache-keyable** — which is exactly what makes what-if cheap (Section 13).

---

8. Stage Intelligence Architecture

8.1 Terminology discipline

The brief demands precision. This project uses these words with exactly these meanings:

Term	Definition in this project	Where used
AI/ML model	A fitted statistical artifact producing a numeric output from features	Fusion model, stage sub-models, anomaly detector, calibrator
Intelligence module	A named component that evaluates one manufacturing stage by combining rules, derived indicators, statistics, and (optionally) a small model, producing health/risk/anomaly output. **Stateless, deterministic, non-autonomous.**	The five stage modules
Service	A process-level component exposing an interface to other components (inference service, analytics service, simulation service)	Application layer
Processing component	A pure data transformation (scaler, encoder, feature builder)	Pipeline
Autonomous agent	A component that sets its own goals, chooses actions, and acts on the environment without a scripted control flow	**Deliberately NOT used in this project**

Why no agents. The control flow of an injection-molding cycle is fixed and known: material, plasticize, inject, pack, cool, inspect. There is no planning problem, no action selection, no environment to act upon. Introducing an "agent framework" here would be decoration that adds latency, nondeterminism, and API cost while removing testability. The one place where an agent-like pattern could be *argued* for is the recommendation search (goal-directed exploration under constraints) — and even there, a deterministic constrained optimizer is strictly better: reproducible, explainable, and bounded. **This document therefore recommends zero autonomous agents.** Saying this explicitly, with reasons, is itself a defensible architectural contribution.

8.2 Stage intelligence specification table

Stage	Inputs	Intelligence method	Outputs	Risk basis
S1 Material Conditioning	moisture_pct, mfi, regrind_pct, drying_time_h, dryer_temp_c, resin_grade, colourant_pct	Spec-limit rules + z-score vs lot history + derived `dryness_margin`	material_health, moisture_risk, viscosity_class (low/nominal/high)	Breach of resin spec sheet limits; deviation from lot distribution
S2 Plasticizing / Thermal	barrel_zone_temps[3-4], nozzle_temp, melt_temp, screw_rpm, back_pressure, residence_time_s, (from S1: viscosity_class)	Rules + derived `thermal_exposure = f(melt_temp, residence_time)` + profile-monotonicity check + small regression for melt-temp consistency	thermal_health, degradation_exposure, melt_consistency, temp_profile_anomaly	Degradation ceiling, profile inversion, shear-heat proxy
S3 Injection / Fill	inj_velocity_mm_s, inj_pressure_bar, fill_time_s, shot_size, switchover_position, (from S2: effective_melt_temp, viscosity proxy)	Rules + derived `fill_index = pressure x velocity / viscosity_proxy` + supervised short-shot/flash risk sub-model	fill_health, short_shot_risk, flash_risk, shear_rate_proxy	Fill index outside band; pressure near machine max; velocity x temp joint breach
S4 Packing / Holding	hold_pressure_bar, hold_time_s, cushion_mm, (from S3: fill completion proxy), thickness_mm	Rules + derived `packing_integral = hold_pressure x hold_time` + gate-freeze heuristic	packing_health, sink_void_risk, over_pack_risk	Packing integral below/above documented band; cushion out of range
S5 Cooling / Ejection	mold_temp_c, cooling_time_s, thickness_mm, ejection_force_proxy, cycle_time_s	Rules + derived `cooling_adequacy = cooling_time / required_time(thickness, mold_temp)` + warp-risk sub-model	cooling_health, warp_risk, ejection_risk, dimensional_stability_index	Cooling adequacy < 1; mold temp imbalance; cycle inconsistency
S6 Quality (Fusion)	all raw params + all stage vectors + engineered interactions + machine/product context	Gradient-boosted trees + probability calibration + SHAP	PASS/FAIL, p_fail, risk level, attributions	Calibrated probability + anomaly + constraint breaches (Section 12)

8.3 Per-stage detailed specification

Each stage is documented against the ten required fields.

S1 — MATERIAL CONDITIONING

1. **Purpose.** Establish whether the incoming resin lot is fit to process, and produce a viscosity expectation that downstream stages consume.
2. **Inputs.** `moisture_pct`, `mfi` (melt flow index), `regrind_pct`, `drying_time_h`, `dryer_temp_c`, `resin_grade`, `colourant_pct`.
3. **Processing logic.**
   - Rule: `moisture_pct <= spec_max(resin_grade)` (PP typically very low; hygroscopic grades stricter).
   - Rule: `regrind_pct <= plant_policy_max` (commonly 20–30% for non-critical parts).
   - Derived: `dryness_margin = (spec_max - moisture) / spec_max`.
   - Derived: `viscosity_class` from MFI tertiles of the training distribution — LOW_MFI => high viscosity => downstream needs more pressure/temperature.
   - Statistical: z-score of MFI and moisture against the lot-history distribution for that resin grade.
4. **Model/rule/statistics.** Rule engine + z-score statistics. **No ML model.** Justification: the relationships here are specification-driven, not learned; a model would add opacity without accuracy.
5. **Output.** `{material_health, moisture_risk, viscosity_class, dryness_margin, mfi_z, notes[]}`.
6. **Risk score.** `1 - material_health`, where health is a weighted penalty aggregation over rule violations and |z| magnitudes, clipped to [0,1].
7. **Anomaly status.** True if any |z| > 3 or a hard spec breach occurs.
8. **State transition.** COMPLETED normally; BLOCKED if moisture exceeds spec by more than a configured critical factor.
9. **Effect on next stage.** Publishes `viscosity_class` and `moisture_flag` into upstream context. THERMAL uses `viscosity_class` to adjust its expected melt-temperature band; it also raises splay risk if `moisture_flag` is set and melt temp is high.
10. **Failure mode it predicts.** Splay/silver streaking, inconsistent shot weight, property degradation from excessive regrind.

S2 — PLASTICIZING / THERMAL

1. **Purpose.** Evaluate whether the melt is being prepared within a thermally safe, consistent window.
2. **Inputs.** `barrel_temp_rear/mid/front`, `nozzle_temp`, `melt_temp`, `screw_rpm`, `back_pressure_bar`, `residence_time_s`, upstream `viscosity_class`, `moisture_flag`.
3. **Processing logic.**
   - Rule: melt temp within the resin's documented processing window.
   - Rule: barrel profile should be monotonic non-decreasing rear -> front (an inverted profile is a recipe error worth flagging).
   - Derived: `thermal_exposure = normalized(melt_temp_excess) x normalized(residence_time)` — a documented proxy for degradation risk; this is the classic time-at-temperature interaction.
   - Derived: `shear_heat_proxy = f(screw_rpm, back_pressure)` — elevated RPM + back pressure raises effective melt temperature beyond setpoint.
   - Derived: `effective_melt_temp = melt_temp + shear_heat_offset`.

- Sub-model (optional, P1): small regression predicting expected melt temp from the barrel profile; a large residual signals sensor/setpoint inconsistency.

4. **Method.** Rules + derived indicators + optional ridge regression consistency check.
5. **Output.** `{thermal_health, degradation_exposure, effective_melt_temp, melt_consistency, temp_profile_anomaly, notes[]}`.
6. **Risk score.** Weighted aggregation dominated by `degradation_exposure` and window breaches.
7. **Anomaly.** Profile inversion, or consistency residual beyond training-set 99th percentile.
8. **State transition.** BLOCKED if `melt_temp > degradation_ceiling(resin_grade)`.
9. **Effect on next stage.** Publishes `effective_melt_temp` and a `viscosity_proxy` derived from MFI + effective temperature. INJECTION's fill index divides by this proxy — this is the explicit encoding of the temperature x pressure coupling.
10. **Failure modes.** Burn marks, degradation, brittleness, discolouration, and (jointly with moisture) splay.

S3 — INJECTION / FILL

1. **Purpose.** Assess whether the cavity will fill completely without flashing or shear damage.
2. **Inputs.** `inj_velocity_mm_s`, `inj_pressure_bar`, `fill_time_s`, `shot_size_mm`, `switchover_position_mm`, `clamp_force_ton`, upstream `effective_melt_temp`, `viscosity_proxy`.
3. **Processing logic.**
   - Derived: `fill_index = (inj_pressure x inj_velocity) / viscosity_proxy` — normalised so that ~1.0 means "expected to fill cleanly".
   - Rule: `inj_pressure <= 0.9 x machine_max_pressure` (headroom rule; running at the machine ceiling means no compensation capacity).
   - Rule: `clamp_force >= projected_area x cavity_pressure_estimate` (flash guard).
   - Derived: `shear_rate_proxy` from velocity and gate geometry constant.
   - Sub-model: gradient-boosted binary classifier for `short_shot_or_flash_risk`, trained on the labelled defect-mode column if available, otherwise a rule-derived pseudo-target (documented as such, and excluded from the fusion target to avoid leakage).
4. **Method.** Rules + derived indicators + a small supervised sub-model.
5. **Output.** `{fill_health, short_shot_risk, flash_risk, fill_index, shear_rate_proxy, notes[]}`.
6. **Risk score.** `max(short_shot_risk, flash_risk)` blended with rule penalties.
7. **Anomaly.** `fill_index` outside the training-set 1st–99th percentile band.
8. **State transition.** BLOCKED on clamp-force violation (a genuine machine-safety class of breach).
9. **Effect on next stage.** Publishes `fill_completion_proxy` and `cavity_pressure_estimate`; PACKING uses these to decide whether the packing phase starts from a full or partially filled cavity.
10. **Failure modes.** Short shot, flash, jetting, shear splay, weld-line weakness.

S4 — PACKING / HOLDING

1. **Purpose.** Assess compensation for volumetric shrinkage before gate freeze.
2. **Inputs.** `hold_pressure_bar`, `hold_time_s`, `cushion_mm`, `thickness_mm`, upstream `fill_completion_proxy`, `cavity_pressure_estimate`, `effective_melt_temp`.
3. **Processing logic.**
   - Derived: `packing_integral = hold_pressure x hold_time` (a documented first-order proxy for total compensation delivered).
   - Derived: `gate_freeze_estimate = g(thickness_mm, mold_temp, effective_melt_temp)` using a documented heuristic constant, **explicitly labelled as a heuristic, not a solved physical time**.
   - Rule: `hold_time` beyond `gate_freeze_estimate` is wasted cycle time (economic flag, not quality flag).
   - Rule: `hold_time` far below gate freeze => sink/void risk.
   - Rule: `cushion_mm` within [min, max] — zero cushion means loss of pressure control; excessive cushion means excessive residence.
4. **Method.** Rules + derived indicators. Optional P1 sub-model for sink-mark risk.
5. **Output.** `{packing_health, sink_void_risk, over_pack_risk, packing_integral, gate_freeze_estimate, notes[]}`.
6. **Risk score.** Distance of `packing_integral` from the learned optimal band, signed (under-pack vs over-pack are different failure modes and must not be averaged).
7. **Anomaly.** Cushion at or near zero; packing integral outside percentile band.
8. **State transition.** Normal; no hard block (over-packing is a quality problem, not a safety one, unless clamp is exceeded).
9. **Effect on next stage.** Publishes `residual_stress_proxy` (high over-pack => higher warp tendency), consumed by COOLING.
10. **Failure modes.** Sink marks, voids, dimensional undersize, flash (over-pack), ejection difficulty (over-pack).

S5 — COOLING / EJECTION

1. **Purpose.** Assess whether the part solidifies adequately and uniformly before ejection.
2. **Inputs.** `mold_temp_c`, `cooling_time_s`, `thickness_mm`, `cycle_time_s`, `ejection_force_proxy`, upstream `residual_stress_proxy`, `effective_melt_temp`.
3. **Processing logic.**
   - Derived: `required_cooling_estimate = h(thickness_mm, mold_temp, melt_temp)` — documented heuristic (thickness-squared family), explicitly flagged as heuristic.
   - Derived: `cooling_adequacy = cooling_time / required_cooling_estimate`.
   - Rule: `cooling_adequacy < 1.0` => warp/dimensional risk.
   - Rule: mold temp outside coolant-system band => surface finish and warp risk.

    - Derived: `cycle_consistency = cycle_time vs sum(stage times)` — inconsistency indicates an unmodelled delay.
    - Sub-model: warp-risk classifier if warp-labelled data exists.
4. **Method.** Rules + derived indicators + optional sub-model.
5. **Output.** `{cooling_health, warp_risk, ejection_risk, cooling_adequacy, dimensional_stability_index, notes[]}`.
6. **Risk score.** Driven by `cooling_adequacy` shortfall and `residual_stress_proxy`.
7. **Anomaly.** Cycle-time inconsistency; mold temp far from machine-profile norm.
8. **State transition.** COMPLETED -> QUALITY_INSPECTION.
9. **Effect on next stage.** All five stage vectors are handed to the fusion engine.
10. **Failure modes.** Warpage, dimensional deviation, ejection marks, part sticking, post-mold shrinkage.

8.4 Health score aggregation contract

Every stage produces `health_score ∈ [0,1]` through the same documented formula shape so the UI and fusion engine can treat them uniformly:

```
penalty = SUM_over_checks( weight_i * severity_i )
health  = clip(1 - penalty, 0, 1)

severity_i for a range check =
    0                                  if value inside [lo, hi]
    min(1, distance_outside / tolerance) if outside

severity_i for a z-score check =
    min(1, max(0, (|z| - 2) / 2))
```

Weights are stored in the same versioned constraint registry as the ranges. They are **documented domain assumptions**, not learned parameters, and the document must say so. A P1 enhancement is to fit the weights by regressing stage health against observed failure outcomes — but only if enough data exists, and only as a clearly-labelled second model version.

8.5 Why stage outputs are not the only input to the fusion model

Stage modules compress information. Compression loses signal. If the fusion model saw only five health scores, it could never learn an interaction that the module designers did not anticipate. Section 10 defines the fusion input as raw + stage + engineered, precisely to keep the learned model free to discover what the hand-built rules missed.

---

9. ML / DL Architecture

9.1 End-to-end pipeline

```
[ Raw dataset ]
      |
      v
(1) INGESTION            csv/parquet loader, schema contract, source manifest
      |
      v
(2) VALIDATION           column presence, dtype, range, nullity, duplicate keys
      |                  -> validation report artifact; hard fail on contract breach
      v
(3) EDA                  univariate, bivariate, target-conditioned distributions,
      |                  correlation, VIF, class balance, machine-wise failure rates
      v
(4) CLEANING             de-duplication, impossible-value removal, unit harmonisation
      |
      v
(5) MISSING VALUES       strategy per column (below)
      |
      v
(6) OUTLIERS             IQR + robust z; flag, do not blindly delete
      |
      v
(7) ENCODING             machine_id / resin_grade / tool_id -> target or one-hot
      |
      v
(8) SCALING              standardise for linear/NN; trees need none (kept in pipeline
      |                  anyway so one preprocessing object serves all models)
      v
(9) FEATURE ENGINEERING  physics-motivated derived features + interactions
      |
      v
(10) SPLIT               grouped + temporal split (critical, see 9.6)
      |
      v
(11) IMBALANCE           class weights first; SMOTE only inside CV folds if needed
      |
      v
(12) TRAINING            baseline -> tree ensembles -> tuned ensemble -> (MLP)
      |
      v
(13) TUNING              stratified/grouped CV + Optuna or randomised search
      |
      v
(14) EVALUATION          PR-AUC first, recall on FAIL, calibration, confusion matrix
      |
      v
(15) CALIBRATION         isotonic / Platt on a held-out calibration split
      |
      v
(16) SERIALIZATION       joblib bundle: preprocessor + model + calibrator + metadata
      |
      v
(17) INFERENCE           single-record and batch paths, identical feature builder
      |
      v
(18) EXPLAINABILITY      SHAP TreeExplainer, precomputed background sample
      |
      v
(19) MONITORING          prediction logging, input drift stats, performance snapshot
```

9.2 Missing-value policy (per column class)

Column class	Policy	Rationale
Setpoint parameters (temps, pressures, times)	Median within `(machine_id, resin_grade)` group; add `_was_missing` flag	Setpoints are machine-specific; global median distorts
Material lot attributes	Median within resin grade; flag	Lot properties cluster by grade
Derived sensor readings	Forward-fill only if a genuine time index exists; otherwise group median	Avoid inventing temporal structure
Categorical IDs	Explicit `UNKNOWN` category	Missingness can itself be informative
Target	**Drop the row.** Never impute a label	Label imputation is fabrication

Rule: **all imputation is fitted inside the pipeline on the training fold only.** Imputing before splitting is a leakage vector and is explicitly forbidden.

9.3 Feature engineering catalogue

Physics-motivated, each with a written justification — this is what separates this project from a generic AutoML submission.

Feature	Definition	Why
`viscosity_proxy`	f(MFI, effective_melt_temp)	Encodes material-temperature coupling
`effective_melt_temp`	melt_temp + shear_heat_offset(rpm, back_pressure)	Setpoint != actual melt condition
`thermal_exposure`	melt_temp_excess x residence_time	Degradation is time-at-temperature
`fill_index`	(inj_pressure x inj_velocity) / viscosity_proxy	Core temperature-pressure interaction
`packing_integral`	hold_pressure x hold_time	Total compensation delivered
`cooling_adequacy`	cooling_time / required_cooling_estimate	Normalises time by thickness/temp
`pressure_headroom`	1 - inj_pressure / machine_max_pressure	Control capacity remaining
`temp_profile_slope`	linear fit across barrel zones	Detects inverted/flat profiles
`cycle_balance`	cooling_time / cycle_time	Cycle composition signature
`moisture_temp_risk`	moisture_pct x melt_temp	Splay mechanism
`machine_age_norm`	machine age / fleet median	Machine context without ID leakage
`days_since_maintenance`	context	Known driver of drift
`regrind_mfi_interaction`	regrind_pct x mfi	Regrind shifts effective rheology
Rolling stats (if temporal)	rolling mean/std of key params over last N shots per machine	Captures drift, not just setpoint

Rolling features are a leakage hazard. They may only use strictly past shots, computed after a temporal split, never across the split boundary.

9.4 Encoding decisions

- `machine_id` — target/impact encoding with **out-of-fold** computation, plus smoothing. Rationale: 5–20 machines; one-hot is fine too and safer. Recommendation: **one-hot for MVP**, target encoding as a tuned P1 experiment with leakage-safe implementation.
- `resin_grade` — one-hot (few levels).
- `shift`, `operator_id` — deliberately **excluded**. They correlate with outcomes but give the recommendation engine nothing actionable and invite spurious attributions. Document the exclusion.

9.5 Candidate models and the deep-learning question

Model	Role	Expected behaviour	Verdict
Logistic Regression	Baseline, interpretability floor	Captures monotone effects, fails on interactions like over/under-packing	**Include as baseline**
Decision Tree	Interpretable baseline, rule extraction	Overfits alone, but the extracted rules are useful for the report	**Include**
Random Forest	Robust ensemble	Strong, low-tuning, good feature importances	**Include**
XGBoost / LightGBM	Primary candidate	Best-in-class on tabular, handles interactions and missing values, fast SHAP	**Primary model**
MLP (1–3 hidden layers)	DL comparison	Competitive only with careful scaling and enough rows; usually a few points behind GBDT	**Include as benchmarked comparison**
1-D CNN / LSTM	Sequence modelling	Only meaningful if per-shot time-series curves exist (cavity pressure traces)	**Conditional** (see below)
TabNet / FT-Transformer	Modern tabular DL	High complexity, marginal gain at this data scale	**Excluded**

On “the PS says Deep Learning”. The academically correct answer is not to force a neural network onto a 10–50k-row tabular problem and pretend it is superior. It is to:

1. Train GBDT and MLP under the identical pipeline and split.
2. Report both, with confidence intervals across CV folds.
3. State plainly *why* tree ensembles typically win on tabular data: axis-aligned splits match threshold-like process limits; they are invariant to monotone feature scaling; they handle mixed types and missingness natively; and they are far more sample-efficient than networks at this size.
4. Then incorporate deep learning **where it is genuinely the right tool**:
    - **Autoencoder for anomaly detection** (Section 12) — unsupervised reconstruction error on the process vector. This is a legitimate DL contribution with a real job.
    - **1-D CNN over cavity-pressure / screw-position curves**, *if and only if* curve data exists or is credibly synthesised. A pressure trace is a sequence; that is where DL earns its place.
    - **Embedding layer for machine/tool identity** inside the MLP, demonstrating learned categorical representations.

This framing — "DL used where it is justified, benchmarked honestly where it is not" — is a stronger result than a forced neural network, and it is defensible in the viva.

9.6 Splitting strategy (the most important ML decision in this project)

A naive random split will inflate metrics badly. Three leakage vectors exist:

1. **Batch leakage.** Consecutive shots from the same batch share setpoints and lot. Random splitting puts near-duplicates on both sides.
2. **Temporal leakage.** Process drifts. Training on the future and testing on the past overstates real performance.
3. **Machine leakage.** If evaluating generalisation to unseen machines, machine-wise grouping is required.

Recommended protocol:

- Primary: **GroupKFold by** `batch_id` for model selection (5 folds, stratified where possible).
- Final holdout: **temporal** — last ~20% of the time index as an untouched test set.
- Additional diagnostic: **leave-one-machine-out** to report cross-machine generalisation separately. Expect worse numbers; report them anyway. Honest degradation reporting is a credibility asset.
- Calibration set carved from the training portion, never from the test set.

9.7 Class imbalance

Real inspection data is typically 90–98% PASS. Order of escalation:

1. `class_weight='balanced'` / `scale_pos_weight` — first choice, no data fabrication.
2. Threshold tuning on the validation PR curve — the real fix for imbalance is usually the decision threshold, not resampling.
3. SMOTE / SMOTENC — only inside CV folds, only if 1 and 2 are insufficient, and reported as an ablation. Never applied before splitting.
4. Focal loss for the MLP variant.

Metric discipline: accuracy is reported but never used for selection. **PR-AUC and recall on the FAIL class are the selection metrics.**

9.8 Hyperparameter tuning

- Search: Optuna TPE, 50–150 trials, with an explicit time budget.
- Objective: mean PR-AUC across GroupKFold folds.
- Guardrails: fixed random seeds, logged search space, early stopping on the validation fold.
- Every trial logged to MLflow (or a CSV fallback) with parameters, metrics, and the git commit hash.

9.9 Calibration

Raw GBDT probabilities are not well calibrated, and this project *uses probabilities semantically* (risk bands, recommendation deltas). Therefore calibration is mandatory, not optional.

- Fit isotonic regression (or Platt, if the calibration set is small) on a held-out calibration split.
- Report a reliability diagram and Brier score before and after.
- Persist the calibrator inside the model bundle so inference and training cannot diverge.

9.10 Serialization contract

A model version is a single directory:

```
models/fusion/v1.3.0/
  +-- preprocessor.joblib        fitted ColumnTransformer
  +-- model.joblib               fitted estimator
  +-- calibrator.joblib
  +-- shap_background.parquet    sampled background rows for the explainer
  +-- feature_schema.json        ordered feature names + dtypes + ranges
  +-- metrics.json               full evaluation report
  +-- metadata.json              git sha, dataset hash, train date, split spec, seed
  +-- constraints_version.txt    which constraint registry version it was trained with
```

Inference loads the **bundle**, never individual pieces. Feature order is read from `feature_schema.json`, never hard-coded — this eliminates the single most common serving bug.

---

10. Quality Prediction — The Fusion Engine

10.1 Fusion input composition

```
+------------------------------+
| A. Raw process parameters    |  ~18 columns: all setpoints and lot attributes
+------------------------------+
| B. Stage intelligence outputs |  5 x {health, risk_ordinal, anomaly_score,
|                              |       2-3 stage-specific indicators} ~ 25 columns
+------------------------------+
| C. Engineered features       |  ~14 physics-motivated derived features
+------------------------------+
| D. Machine / product context  |  machine one-hot, age, maintenance gap,
|                              |  cavity count, thickness, nominal shot weight
+------------------------------+
                |
                v
        +-----------------+
        |  FUSION MODEL   |  XGBoost / LightGBM (primary)
        |  + calibrator   |
        +--------+--------+
                 |
       +---------+----------+
       |                    |
  PASS / FAIL          p_fail (calibrated)
       |                    |
       +---------+----------+
                 v
          RISK ENGINE (Sec. 12)
```

10.2 Why raw features must be preserved

1. **Information bottleneck.** Stage health scores are rule-weighted compressions designed by humans. Any interaction the designers did not encode is destroyed by that compression. The model must be able to find `melt_temp x cooling_time` effects even though no single stage owns that pair.
2. **Explainability quality.** Attributions over raw, named parameters ("melt temperature contributed +0.18 to failure probability") are actionable. Attributions over an abstract "thermal_health" are not — the user cannot turn a knob labelled thermal_health.
3. **Recommendation feasibility.** The recommendation engine searches over *actionable* variables. Those must be model inputs, otherwise the search cannot move the prediction.
4. **Debuggability.** If a stage rule is wrong, a raw-feature-inclusive model degrades gracefully instead of inheriting the error wholesale.

10.3 The stage-output leakage caution

Stage outputs are **deterministic functions of the raw inputs**. Including both raw and derived versions is legitimate (it is feature engineering), but two cautions apply:

- **Multicollinearity** is expected and harmless for trees; it does distort SHAP attributions by splitting credit across correlated features. The report must note this and prefer SHAP interaction values or grouped attributions for headline claims.
- If any stage sub-model was trained using the *final quality label*, then feeding its output into the fusion model **trained on the same rows** is target leakage. **Mitigation:** stage sub-models must be trained with out-of-fold predictions (stacking discipline), i.e. the stage output for row `i` is produced by a model that never saw row `i`. This is a strict requirement, not a nicety.

10.4 Fusion model output contract

```
PredictionResult
  label            : "PASS" | "FAIL"
  p_fail           : float, calibrated, [0,1]
  threshold_used   : float (tuned, not 0.5 by default)
  model_version    : str
  risk_level       : NORMAL|LOW|MEDIUM|HIGH|CRITICAL
  risk_drivers[]   : {source: model|anomaly|constraint, detail, weight}
  stage_risk_mass[]: per-stage share of the total attributed failure risk
  explanation      : local SHAP attributions (top-k, signed)
  uncertainty      : {method: "ensemble_spread"|"none", value}
  caveats[]        : e.g. "input outside training distribution on 2 features"
```

The `caveats` field is important: if an input sits outside the training range, the prediction is an extrapolation and the UI must say so rather than showing a confident number.

---

11. Explainable AI

11.1 Techniques and where each is used

Technique	Scope	Where surfaced	Cost
SHAP TreeExplainer (local)	Single prediction	Explainability panel after every run	Milliseconds with a sampled background
SHAP global (mean abs)	Whole model	Model insight page, report	Precomputed offline
SHAP dependence plots	Feature effect shape	Report + analytics page	Offline
SHAP interaction values	Pairwise effects	Report only (expensive)	Offline, sampled
Permutation importance	Model-agnostic sanity check	Report; cross-check against SHAP	Offline
Partial dependence / ICE	Effect shape	Report + optional sensitivity view	Offline
1-D parameter sweeps	Live sensitivity	What-if sensitivity strip in UI	~50 inference calls, cached
Decision-tree surrogate	Global rule summary	Report ("the model in 12 rules")	Offline
Stage risk mass	Which stage drives risk	Main UI, per-stage badges	Derived from SHAP grouping

11.2 Stage risk mass — a project-specific explainability construct

Raw SHAP gives per-feature attributions. Users think in stages. Therefore:

```
stage_risk_mass(S) = SUM of |shap_value(f)| for all features f owned by stage S
                     ------------------------------------------------------
                     SUM of |shap_value(f)| over all features
```

Feature-to-stage ownership is declared in the feature schema. Every engineered feature is assigned to the stage whose parameters dominate it (`fill_index` -> INJECTION; `thermal_exposure` -> THERMAL). Cross-stage features (e.g. `moisture_temp_risk`) are split 50/50 between their parent stages, and the split rule is documented.

This produces the headline UI line: *"68% of the predicted failure risk originates in the Cooling stage."* It is intuitive, it maps to the visualization, and it is derived transparently.

11.3 Presentation format

```
PREDICTION: FAIL        P(fail) = 0.71 (calibrated)     Risk: HIGH

Top contributing factors (SHAP, signed, toward FAIL):

  cooling_time (19 s)          ########################   +0.21
  cooling_adequacy (0.63)      ####################       +0.16
  melt_temp (278 C)            ###############            +0.12
  packing_integral (low)       ###########                +0.09
  moisture_pct (0.09%)         #######                    +0.06
  machine MC-07                ###                        +0.02
  ------------------------------------------------------------
  inj_velocity (112 mm/s)      #####                      -0.04  (toward PASS)

Risk originates mainly in: COOLING (58%), THERMAL (24%), PACKING (11%)
```

11.4 Limitations that must be stated in the UI and the report

1. **SHAP is attribution, not causation.** It decomposes *this model's* output, not the physical process. If the model learned a spurious correlation, SHAP will faithfully explain the spurious correlation.
2. **Correlated features split credit.** With `melt_temp`, `effective_melt_temp`, and `thermal_exposure` all present, individual attributions understate the true importance of "temperature" as a concept. Grouped attribution is the honest presentation.
3. **Background-set dependence.** SHAP values are relative to a chosen background distribution; changing it changes the numbers. The background sample is versioned with the model.
4. **Additivity is local.** Attributions do not compose into a global causal story, and summed attributions across runs are not a process diagnosis.
5. **Out-of-distribution inputs produce meaningless attributions.** The `caveats` flag gates the panel: if the input is an extrapolation, the UI shows a warning banner above the explanation.

A short, permanently visible footnote in the explainability panel: *"Attributions describe how this model reached its output for these inputs. They are statistical associations learned from historical data, not verified physical causes."*

---

12. Anomaly Detection

12.1 Does it earn its place?

Yes, but for a specific reason. The supervised fusion model can only recognise failure patterns **present in the training labels**. A novel process condition — a new resin lot behaving oddly, a machine drifting after a repair, an operator entering a physically strange combination — may be scored confidently by the classifier simply because it is far from anything it has seen. Unsupervised anomaly detection is the guard against unwarranted confidence.

12.2 Where it lives — a three-level design

Level	What it does	Method	Justification
L1: Within each stage	Univariate/small-slice outlier flags on the stage's own parameters	Robust z-score, IQR, spec-limit checks, EWMA control-chart logic for temporal drift	Cheap, interpretable, gives the UI per-stage anomaly badges
L2: Global process vector	Detects unusual *combinations* across the whole recipe	**Isolation Forest** (primary) fitted on the PASS-dominant training distribution	Fast, no scaling sensitivity, handles mixed distributions, gives a usable score
L3: Deep reconstruction (P1)	Learns the normal manifold; large reconstruction error = novel condition	**Autoencoder** on scaled numeric features	This is the legitimate deep-learning contribution; also enables per-feature reconstruction error, which localises the anomaly

Rejected: One-Class SVM — comparable behaviour to Isolation Forest but scales poorly and is more sensitive to kernel/scaling choices; keep it as a reported baseline comparison only.

12.3 Why not fold anomaly detection into the fusion model

Three reasons: 1. **Different question.** The classifier answers "will this fail?"; the detector answers "have we seen anything like this before?". Confusing them is exactly how systems produce confident nonsense. 2. **Different supervision.** The detector trains unsupervised on normal-operation data; it stays valid even when failure labels are scarce. 3. **Different consumer.** The risk engine treats a high anomaly score as a reason to *widen* uncertainty and *downgrade* trust in the classifier — a role that a single blended score cannot perform.

12.4 Statistical process control layer

If the dataset has a usable time index, add lightweight SPC per machine and parameter:

- X-bar / moving-range style control limits computed from a stable reference window.
- Western-Electric-style rule subset: 1 point beyond 3 sigma; 8 consecutive points on one side of the mean; 2 of 3 beyond 2 sigma.
- Surfaced on the analytics page as drift alerts, and as a note in the run timeline.

This is cheap, classic, and immediately legible to any manufacturing-literate examiner.

12.5 Anomaly output contract

```
AnomalyResult
  scope        : "stage:THERMAL" | "global"
  method       : "isolation_forest" | "zscore" | "autoencoder" | "spc"
  score        : float (normalised 0..1, higher = more anomalous)
  is_anomaly   : bool (threshold from training-set percentile, e.g. top 2%)
  top_contributors[] : features with largest deviation / reconstruction error
  note         : human-readable
```

---

13. What-If Simulation

13.1 Definition

A what-if is a **branch**: a child run derived from a parent run with one or more parameters mutated, executed through the identical stage pipeline and fusion model, and presented as a diff against the parent.

13.2 Computational strategy: partial re-simulation via a dependency graph

Full re-simulation of every stage is only ~5–20 ms, so correctness could be bought by always re-running everything. But partial re-simulation is worth implementing because (a) it is the technically interesting answer the brief asks for, and (b) it makes the live-slider experience and the recommendation search (hundreds of evaluations) comfortably fast.

Parameter -> stage dependency graph:

```
moisture_pct, mfi, regrind_pct, drying_* --> S1 --> S2 --> S3 --> S4 --> S5 --> FUSION
barrel_temps, melt_temp, screw_rpm, back_pressure -----> S2 --> S3 --> S4 --> S5 --> FUSION
inj_velocity, inj_pressure, fill_time, switchover -------------> S3 --> S4 --> S5 --> FUSION
hold_pressure, hold_time, cushion ----------------------------------> S4 --> S5 --> FUSION
mold_temp, cooling_time, cycle_time ------------------------------------------> S5 --> FUSION
machine_id, thickness, cavity_count ------> (context: affects S3,S4,S5 and FUSION)
```

Invalidation rule. Changing a parameter invalidates the earliest stage that consumes it and every stage downstream of it. Stages upstream of the change are reused from the parent run's cached results. The fusion model always re-runs (it consumes everything).

Example: changing `cooling_time` invalidates only S5 and FUSION. S1–S4 results are reused verbatim. Example: changing `melt_temp` invalidates S2 onward — almost the whole chain, correctly, because melt temperature propagates through viscosity into fill and packing behaviour.

13.3 Caching design

- **Cache key:** `hash(stage_name, stage_input_slice, upstream_context_slice, module_version, constraints_version)`.
- **Store:** in-process LRU dict (per server instance), size-bounded (e.g. 5,000 entries). No Redis — unjustified at this scale.
- Because stage modules are pure functions, this cache is trivially correct. Purity is enforced by a unit test that calls each module

twice with identical inputs and asserts identical outputs.
- Fusion predictions are cached on the full feature vector hash + model version.

13.4 Modes of what-if

Mode	Interaction	Implementation
Single-parameter mutation	User edits one field, presses Evaluate	Partial re-sim, one inference
Live slider (debounced)	User drags a slider; result updates continuously	Debounce ~150 ms, partial re-sim, cache hits dominate
Multi-parameter recipe edit	User edits several fields then evaluates	Partial re-sim from the earliest invalidated stage
1-D sensitivity sweep	System sweeps one parameter across its admissible range	~40 evaluations, run in one request, returns a curve
2-D response surface (P1)	Two parameters swept on a grid	20x20 = 400 evaluations; run async, render heatmap
Scenario A/B compare	Two saved configurations side by side	Two stored runs, diffed client-side

13.5 Sensitivity analysis surface

For the currently selected run, the system computes for each actionable parameter:

- `local_gradient` — finite-difference change in `p_fail` per unit change, at the current operating point.
- `range_swing` — `max(p_fail) - min(p_fail)` across the parameter's admissible window with all else fixed.
- `flip_point` — the nearest value (if any) at which the predicted label flips.

This produces a ranked "which knob matters most right here" list, which is *different from SHAP* (SHAP explains the current attribution; sensitivity explains the local responsiveness). Presenting both, and explaining the difference, is a strong technical point for the report.

13.6 Diff presentation contract

```
                        CURRENT (A)           PROPOSED (B)          DELTA
Melt temperature        278 C                 256 C                 -22
Hold pressure           620 bar               660 bar               +40
Cooling time            19 s                  26 s                  +7
-------------------------------------------------------------------------
S1 Material             0.86 LOW              0.86 LOW              unchanged
S2 Thermal              0.61 MEDIUM           0.88 LOW              improved
S3 Injection            0.74 LOW              0.79 LOW              improved
S4 Packing              0.58 MEDIUM           0.81 LOW              improved
S5 Cooling              0.41 HIGH             0.84 LOW              improved
-------------------------------------------------------------------------
Prediction              FAIL                  PASS                  FLIP
P(fail)                 0.71                  0.18                  -0.53
Operational risk        HIGH                  LOW                   -2 bands
Constraint compliance   2 warnings            0 warnings            clean
Cycle time impact       38.0 s                45.0 s                +7.0 s (+18%)
```

Note the last row: any recommendation that lengthens the cycle has an economic cost. Surfacing it prevents the system from looking naive about manufacturing reality, and it is one line of arithmetic.

13.7 State management

- Every what-if creates a **child run** with `parent_run_id` set, so lineage is queryable.
- Child runs are marked `is_whatif = true` and are excluded from analytics aggregates by default (otherwise the failure-rate statistics become meaningless).
- Frontend keeps a draft parameter object in local state; nothing is persisted until Evaluate or Save Scenario is pressed.
- Reset restores the parent's parameters exactly.

---

14. "What Should I Change to Make It Pass?" — Recommendation Engine

14.1 Problem statement

Given a current recipe `x` with `p_fail(x)` high, find a recipe `x'` such that:

```
minimise    p_fail(x')                                (model-predicted failure)
subject to  x'_j in [lo_j, hi_j]          for all j   (admissible process windows)
            x'_j = x_j                    for j in L  (locked / non-actionable params)
            |x'_j - x_j| <= delta_max_j               (max single-step change)
            g_k(x') <= 0                  for all k   (hard domain constraints)
            cost(x, x') minimised         (secondary) (fewest, smallest changes)
```

Hard domain constraints `g_k` include: clamp force sufficiency, melt temperature below the resin degradation ceiling, injection pressure below 90% of machine maximum, cooling time above the thickness-based minimum, and cushion within bounds. **These are enforced as filters, not penalties** — a candidate violating any of them is discarded outright, regardless of predicted probability.

14.2 Approach evaluation

Approach	Description	Pros	Cons	Verdict
A. Learned-space search	Search the model's feature space directly for low-`p_fail` points	Simple, fast	Can produce physically impossible feature combinations if derived features are searched independently	**Partially** — only over *actionable raw* parameters, never derived ones
B. Constrained optimization	Formal optimiser (SLSQP, differential evolution, CMA-ES) with box + inequality constraints	Principled, handles constraints natively	GBDT objective is piecewise-constant -> gradient methods fail; needs derivative-free methods	**Yes, as the refinement stage** (differential evolution)
C. Grid / coordinate search	Discretise each actionable parameter into k levels, evaluate combinations or coordinate-wise	Trivially explainable, deterministic, bounded cost, naturally yields "minimal change" answers	Full grid explodes combinatorially	**Yes, as the primary stage** (coordinate-wise + small local grid)
D. Bayesian optimization	Surrogate model + acquisition function over the search space	Sample-efficient when evaluations are expensive	Our evaluations cost ~5 ms. BO's entire premise (expensive evaluation) does not hold. Adds a dependency and a second model to explain	**Rejected** — unjustified here, and saying why is a better answer than using it
E. Rule + ML hybrid	Domain rules propose the direction of change; ML validates and quantifies the improvement	Physically sensible directions, no absurd suggestions, fully explainable	Requires encoding domain knowledge (which we already have in the constraint registry)	**Yes — this is the governing layer**

14.3 Recommended design: a three-stage funnel

```
STAGE 1 -- RULE-GUIDED CANDIDATE PROPOSAL  (Approach E)
  Input: current recipe + stage risk mass + constraint breaches
  Logic: for each stage flagged MEDIUM+, consult a documented remedy table
         e.g. cooling_adequacy < 1  -> propose increasing cooling_time
                                       and/or reducing mold_temp
              degradation_exposure high -> propose lowering melt_temp
                                       and/or reducing residence_time
              sink_void_risk high  -> propose raising hold_pressure
                                       and/or hold_time (until gate freeze)
  Output: 10-40 directed candidate recipes, all physically sensible by construction

STAGE 2 -- CONSTRAINED LOCAL SEARCH        (Approach C)
  Coordinate-wise search over actionable parameters, k=7 levels each,
  restricted to [x_j - delta_max_j, x_j + delta_max_j] INTERSECT [lo_j, hi_j].
  Two passes: single-parameter moves, then pairwise moves on the top-3
  most sensitive parameters (from Sec 13.5).
  Every candidate passes through the feasibility guard before scoring.
  Output: ranked candidate pool (typically 200-600 evaluations, <3 s).

STAGE 3 -- REFINEMENT                      (Approach B, optional P1)
  Differential evolution seeded from the best Stage-2 candidates,
  bounded by the same windows, with hard constraints as a rejection filter.
  Budget-capped (e.g. 2,000 evaluations / 5 s).
  Output: 3 final recommendations.
```

14.4 The feasibility guard (non-negotiable)

Every candidate — from any stage of the funnel — passes through this gate **before** its predicted probability is ever shown:

```
 1. Box check          : every parameter inside its admissible window
 2. Hard constraints   : clamp force, degradation ceiling, pressure headroom,
                         minimum cooling time, cushion bounds
 3. Coupling checks    : e.g. melt_temp must remain >= mold_temp + minimum_delta;
                         hold_pressure <= injection_pressure
 4. Distribution check : candidate within the convex-ish hull of training data
                         (per-feature percentile bounds + Isolation Forest score).
                         Candidates in regions the model never saw are rejected or
                         flagged EXTRAPOLATED and ranked last.
 5. Step-size check    : no single change exceeds the documented max adjustment
                         (operators do not jump melt temperature by 40 C)
 6. Stability check    : perturb the candidate by +/- small noise 20 times;
                         if predicted label flips in >20% of perturbations,
                         mark UNSTABLE and demote
```

Check 4 and check 6 together are the defence against the most serious failure mode of this feature: **the optimiser exploiting the model rather than improving the process**. A GBDT has flat extrapolation regions and sharp split boundaries; an unconstrained search will find them. Explicitly designing against this — and reporting how often it triggers — is a genuinely strong engineering result.

14.5 Ranking and cost model

Candidates that pass the guard are ranked by a documented composite:

```
score = w1 * (p_fail_current - p_fail_candidate)      # predicted improvement
      - w2 * change_cost(x, x')                        # operational disruption
      - w3 * instability_penalty                       # from check 6
      - w4 * extrapolation_penalty                     # from check 4
      - w5 * cycle_time_increase_normalised            # economic cost
```

`change_cost` uses per-parameter weights: changing a cooling time is cheap; changing a barrel temperature requires thermal stabilisation time; changing a resin lot is expensive. These weights live in the constraint registry with written justification.

14.6 Output presentation

```
AI PROCESS RECOMMENDATION           (model-relative; not plant-validated)

Option 1  -- Minimal change           Predicted P(fail): 0.71 -> 0.24
  Cooling time     19 s -> 26 s
  Rationale: cooling_adequacy was 0.63 (below the thickness-based minimum).
             COOLING carried 58% of attributed risk.
  Cycle time impact: +7.0 s (+18%)
  Constraints: all satisfied. Stability: stable (0/20 flips). In-distribution: yes.

Option 2  -- Balanced                 Predicted P(fail): 0.71 -> 0.12
  Melt temperature 278 C -> 262 C
  Cooling time     19 s  -> 24 s
  Hold pressure    620 bar -> 655 bar
  Rationale: reduces thermal degradation exposure and improves packing integral
             while restoring cooling adequacy.
  Cycle time impact: +5.0 s (+13%)
  Constraints: all satisfied. Stability: stable (1/20 flips). In-distribution: yes.

Option 3  -- Aggressive               Predicted P(fail): 0.71 -> 0.07
  [4 parameter changes]
  Flags: 1 parameter near the edge of the observed training range (EXTRAPOLATED).
         Treat this option as a hypothesis for trial, not a setting to apply.

--------------------------------------------------------------------------------
These options reduce the FAILURE PROBABILITY ESTIMATED BY THIS MODEL for the given
inputs. They are not validated process improvements. Applying them on a real machine
requires a trial run and physical inspection.
```

That closing disclaimer is a required, permanent UI element.

14.7 What the engine must never do

- Never propose a value outside the admissible window, even if `p_fail` is lower there.
- Never propose changes to non-actionable variables (machine ID, part thickness, cavity count) — these are not knobs.
- Never present a recommendation without its constraint-compliance stamp.
- Never claim a real-world quality improvement.
- Never hide the cycle-time or cost consequence.

---

15. Dynamic Interactive Frontend

15.1 Design intent

The main screen should read as a **process control workstation**, not an ML dashboard. Dark industrial surface, high-contrast state colours, monospace numerics for parameter readouts, and a horizontal stage pipeline as the dominant visual element.

15.2 Primary layout — Process Control workspace

```
+=============================================================================+
| SMART MANUFACTURING QUALITY INTELLIGENCE      MC-03 | PP-JBOX-42 | LOT A-771 |
| Run #1042            [ RUN ] [ PAUSE ] [ STEP ] [ RESET ]   speed: [1x  v]   |
+=============================================================================+
|                                                                             |
|   MATERIAL      THERMAL      INJECTION     PACKING      COOLING     QUALITY   |
|   +-------+     +-------+    +-------+     +-------+    +-------+   +------+  |
|   |  [X]  |====>|  [X]  |===>|  [*]  |----:|  [ ]  |---:|  [ ]  |--:|  [ ] |  |
|   | 0.86  |     | 0.61  |    |  ...  |     |       |    |       |   |      |  |
|   |  LOW  |     |MEDIUM |    |ACTIVE |     |PENDING|    |PENDING|   |WAIT  |  |
|   +-------+     +-------+    +-------+     +-------+    +-------+   +------+  |
|                     !                                                       |
|                 anomaly                                                     |
+==============================+==============================================+
| ACTIVE STAGE: INJECTION      | EVENT TIMELINE                               |
|                              |                                              |
| Inputs in scope:             | 12:04:01  run.started                        |
|   Inj. velocity   112 mm/s   | 12:04:02  stage.started    MATERIAL          |
|   Inj. pressure   940 bar    | 12:04:03  stage.completed  MATERIAL  h=0.86  |
|   Fill time       1.42 s     | 12:04:04  stage.started    THERMAL           |
|   Switchover      12.5 mm    | 12:04:06  anomaly.flagged  THERMAL           |
|   (from thermal)             |           residence 41 s @ 278 C             |
|   Eff. melt temp  281 C      | 12:04:06  stage.completed  THERMAL   h=0.61  |
|   Viscosity proxy 0.78       | 12:04:07  stage.started    INJECTION         |
|                              |                                              |
| Derived:                     +==============================================+
|   fill_index      1.06       | PARAMETER PANEL          [edit] [what-if]    |
|   shear proxy     HIGH       |                                              |
|                              | Temperature   [====|=========] 278 C   (!)   |
| Module status:  ANALYSING... | Inj. pressure [=======|======] 940 bar       |
|                              | Inj. velocity [=====|========] 112 mm/s      |
+==============================+ Hold pressure [===|==========] 620 bar       |
| PREDICTION                   | Hold time     [==|===========] 2.1 s         |
|   awaiting quality stage     | Cooling time  [=|============] 19 s    (!)   |
|                              | Mold temp     [======|=======] 42 C          |
|   Risk so far: MEDIUM        |                                              |
+==============================+==============================================+
```

After FINAL_RESULT, the bottom-left panel expands into the result card:

```
+=============================================================================+
| QUALITY RESULT                                                              |
|   Prediction: FAIL         P(fail) 0.71 (calibrated)      Risk: HIGH        |
|   Model: fusion v1.3.0     Threshold: 0.42     In-distribution: 2 warnings  |
|                                                                             |
|   Risk origin:  COOLING 58%  |  THERMAL 24%  |  PACKING 11%  |  other 7%    |
|                                                                             |
|   [ WHY THIS PREDICTION? ]   [ WHAT SHOULD I CHANGE? ]   [ SAVE SCENARIO ]  |
+=============================================================================+
```

15.3 Screen inventory

Screen	Purpose	Key components
Process Control (default)	Run and watch a simulation	Stage pipeline, active-stage panel, timeline, parameter panel, result card
Explainability	Understand a prediction	SHAP bar chart, stage risk donut, caveat banner, dependence plot for the top feature
What-If Workbench	Mutate and compare	Dual-column A/B diff, live sliders, sensitivity strip, flip-point markers
Recommendations	Fix a failing recipe	Option cards, constraint stamps, “send to what-if”, cycle-time impact
History	Browse past runs	Filterable table, run lineage tree (parent/child what-ifs)
Analytics	Aggregate insight	Failure rate by machine / resin lot / parameter bucket, stage risk frequency, SPC drift charts, model metrics
Model Info	Transparency	Active model version, training date, dataset hash, evaluation metrics, calibration plot

15.4 Visual state language

State	Colour	Glyph	Meaning
PENDING	grey	`[ ]`	Not yet reached
ACTIVE	cyan, pulsing	`[*]`	Currently executing
COMPLETED — NORMAL/LOW	green	`[X]`	Healthy
COMPLETED — MEDIUM	amber	`[X]`	Warning
COMPLETED — HIGH	orange	`[X]`	High risk
COMPLETED — CRITICAL	red	`[X]`	Critical
ANOMALY	purple badge `!` overlay	—	Unusual condition, independent of risk
BLOCKED	red outline, halted	`[/]`	Hard constraint breach; requires override

Colour is never the sole carrier of meaning — every state also has a glyph and a text label (accessibility, and it survives projector colour distortion during the demo).

15.5 Technology choices for the frontend

Concern	Choice	Justification	Alternative
Framework	**React 18 + TypeScript + Vite**	Largest ecosystem, type safety catches contract drift against the API, fast dev server	Next.js — rejected: SSR and routing complexity buy nothing for a single-user SPA
Styling	**Tailwind CSS**	Rapid, consistent industrial aesthetic without writing a design system	CSS modules
Component primitives	**shadcn/ui** (Radix)	Accessible dialogs, tabs, sliders, tooltips out of the box	Hand-rolled
Stage pipeline graphic	**Hand-built SVG components**	Full control, trivially animatable, no library weight; the pipeline is 6 nodes and 5 connectors	React Flow — recommended only if free node dragging/zooming is wanted; it is overkill for a fixed linear chain
Charts	**Recharts** (or ECharts)	Declarative, React-native, covers bars/lines/areas/scatter	Plotly (heavier), D3 (build-it-yourself)
Custom viz (SHAP bars, heatmaps)	**Recharts + SVG**	SHAP bars are a horizontal bar chart; the 2-D response surface is a heatmap	D3 only if a bespoke visual is genuinely needed
Server state	**TanStack Query**	Caching, retries, background refresh for run/history endpoints	Manual fetch + useState
Client state	**Zustand**	Tiny store for simulation UI state and draft parameters	Redux Toolkit — too heavy here
Live updates	**Server-Sent Events**	See Section 18	WebSocket, polling
Animation	**CSS transitions + Framer Motion (light use)**	Stage transitions, pulsing active node, value counters	—
3D	**None**	Three.js was evaluated and rejected: a 3D mold adds days of work and zero analytical value	—

16. Backend / API Architecture

16.1 Backend stack

Concern	Choice	Justification
Framework	**FastAPI**	Async, native Pydantic validation (which the constraint registry needs), automatic OpenAPI docs the frontend can typegen from
Server	**Uvicorn**	Standard ASGI server
Validation	**Pydantic v2**	Range validation, custom validators for coupling constraints, clean error envelopes
ORM	**SQLAlchemy 2.0**	Mature, DB-portable (SQLite -> PostgreSQL without code change)
Migrations	**Alembic**	Schema evolution during a multi-week project
ML runtime	scikit-learn, XGBoost/LightGBM, SHAP, joblib	Standard
Background work	**FastAPI BackgroundTasks / asyncio**	No Celery — no broker, no worker fleet, no justification
Config	**pydantic-settings + .env**	Environment-based configuration
Logging	**structlog / stdlib logging, JSON format**	Correlation IDs per request and per run
Testing	**pytest + httpx AsyncClient**	API + unit tests

16.2 Package layout

```
backend/
  app/
    api/
      routes_simulation.py    routes_prediction.py
      routes_whatif.py        routes_recommend.py
      routes_analytics.py     routes_meta.py
      deps.py                 errors.py
    schemas/                  Pydantic request/response models
    services/
      simulation_service.py   inference_service.py
      recommendation_service.py  analytics_service.py
      explain_service.py
    engine/
      state_machine.py        stage_runner.py
      events.py               cache.py
    intelligence/
      base.py                 material.py   thermal.py
      injection.py            packing.py    cooling.py
      anomaly.py              risk_engine.py
    ml/
      features.py             registry.py   predict.py   explain.py
    domain/
      constraints.yaml        remedies.yaml  constraint_loader.py
    db/
      models.py  session.py  repositories.py
    core/
      config.py  logging.py
  ml_pipeline/                offline: training, tuning, evaluation notebooks+scripts
  tests/
  alembic/
```

16.3 API surface

Base path `/api/v1`. All responses use a consistent envelope with `data`, `error`, `meta.request_id`.

Simulation

Method	Path	Purpose
POST	`/simulation/runs`	Create a run from machine + product + material lot + parameters. Validates everything. Returns `run_id` and initial state.
POST	`/simulation/runs/{id}/start`	Begin execution (mode: `stepwise` or `auto`)
POST	`/simulation/runs/{id}/advance`	Execute exactly the next stage (stepwise mode)
POST	`/simulation/runs/{id}/pause`	Pause auto execution
POST	`/simulation/runs/{id}/resume`	Resume
POST	`/simulation/runs/{id}/reset`	Return to INITIALIZED, retain parameters
POST	`/simulation/runs/{id}/override`	Acknowledge a BLOCKED stage and continue (records the override)
GET	`/simulation/runs/{id}`	Full SimulationState
GET	`/simulation/runs/{id}/events`	**SSE stream** of state events
GET	`/simulation/runs/{id}/stages/{stage}`	Single stage detail
GET	`/simulation/runs`	Paginated history with filters

Prediction, what-if, recommendation

Method	Path	Purpose
POST	`/predict`	Stateless one-shot: parameters in, full PredictionResult out (no run persisted)
POST	`/predict/batch`	CSV/JSON array scoring for the analytics page
GET	`/predict/{prediction_id}/explain`	SHAP attributions + stage risk mass
POST	`/whatif`	`{base_run_id, overrides{}}` -> child result + diff
POST	`/whatif/sensitivity`	`{base_run_id, parameter, steps}` -> sweep curve + flip point
POST	`/whatif/compare`	`{run_id_a, run_id_b}` -> structured diff
POST	`/recommend`	`{run_id, locked[], max_changes, objective}` -> ranked options with constraint stamps

Reference and analytics

Method	Path	Purpose
GET	`/machines`, `/products`, `/materials`	Reference entities
GET	`/constraints`	The admissible-window registry (the UI builds sliders from this)
GET	`/models`	Available model versions + metrics
GET	`/models/active`	Currently served version
GET	`/analytics/summary`	Failure rate, run counts, stage risk frequency
GET	`/analytics/by-machine`, `/analytics/by-parameter`	Aggregations
GET	`/health`	Liveness + model-loaded check

16.4 Representative request / response

POST `/api/v1/whatif`

```
Request
{
  "base_run_id": "9f2c...",
  "overrides": { "cooling_time_s": 26.0, "melt_temp_c": 262.0 },
  "persist": true,
  "label": "cooler melt + longer cooling"
}

Response (200)
{
  "data": {
    "run_id": "b71d...",            // child run
    "parent_run_id": "9f2c...",
    "recomputed_stages": ["THERMAL","INJECTION","PACKING","COOLING"],
    "reused_stages": ["MATERIAL"],
    "stage_results": [ ... ],
    "prediction": {
      "label": "PASS", "p_fail": 0.18, "threshold_used": 0.42,
      "risk_level": "LOW", "model_version": "fusion-1.3.0",
      "caveats": []
    },
    "diff": {
      "parameters": [ {"name":"melt_temp_c","from":278,"to":262,"delta":-16}, ... ],
      "stages":    [ {"stage":"COOLING","health_from":0.41,"health_to":0.84,
                      "risk_from":"HIGH","risk_to":"LOW"} ],
      "prediction": {"label_from":"FAIL","label_to":"PASS","p_fail_delta":-0.53},
      "cycle_time": {"from":38.0,"to":45.0,"delta_pct":18.4}
    }
  },
  "error": null,
  "meta": {"request_id":"...","latency_ms":41}
}
```

16.5 Validation and error handling

Three validation tiers:

1. **Schema** (Pydantic): types, required fields, numeric bounds from the constraint registry -> `422` with per-field detail.
2. **Coupling** (custom validators): cross-field rules such as `hold_pressure <= inj_pressure`, `melt_temp >= mold_temp + delta_min` -> `422` with a rule identifier.
3. **Domain/safety** (risk engine at runtime): hard breaches produce a BLOCKED stage, not an HTTP error — the user must see *why* inside the simulation.

Error envelope

```
{ "data": null,
  "error": { "code": "PARAM_OUT_OF_RANGE",
             "message": "melt_temp_c 312 exceeds admissible maximum 295 for PP-H",
             "field": "melt_temp_c",
             "constraint_version": "constraints-2.1",
             "hint": "Reduce melt temperature or select a higher-temperature grade" },
  "meta": {"request_id":"..."} }
```

Error codes are enumerated and stable so the frontend can localise and react to them.

16.6 Authentication

Recommendation: none for the MVP. A single-user college prototype gains nothing from JWT flows and loses days to them. What *is* implemented:

- CORS restricted to the frontend origin.
- Rate limiting on the expensive endpoints (`/recommend`, `/whatif/sensitivity`) via a simple in-process limiter.
- An optional static `X-API-Key` header check, enabled by environment variable, for the deployed demo. Ten lines, sufficient, honest.

This decision is written into the report as a deliberate scope judgement, not an omission.

---

17. Database / Data Model

17.1 Engine choice

- **Development:** SQLite. Zero setup, file-based, ideal for a student team and for CI.
- **Deployment demo:** PostgreSQL via Docker Compose. Same SQLAlchemy models; Alembic handles both.
- No NoSQL. The data is relational, small, and queried relationally. Flexible-schema fields (stage indicator dictionaries, SHAP arrays) are stored as JSON columns — supported natively by both engines.

17.2 Entities (only the justified ones)

```
machines                    products(molds)           materials(lots)
+-- id                      +-- id                    +-- id
+-- code (MC-03)            +-- code                  +-- lot_code
+-- tonnage                 +-- name                  +-- resin_grade
+-- machine_type            +-- cavity_count          +-- mfi
+-- max_inj_pressure        +-- nominal_shot_weight_g +-- moisture_pct
+-- max_clamp_force         +-- wall_thickness_mm     +-- regrind_pct
+-- installed_on            +-- projected_area_cm2    +-- received_on
+-- last_maintenance_on     +-- active                +-- notes
+-- active

process_runs                              run_parameters
+-- id (uuid)                             +-- id
+-- machine_id  -> machines               +-- run_id -> process_runs (1:1)
+-- product_id  -> products               +-- <one column per process parameter,
+-- material_lot_id -> materials          |   ~18 typed numeric/categorical fields>
+-- status (FSM state)                    +-- created_at
+-- mode (auto|stepwise)
+-- is_whatif (bool)                      stage_results
+-- parent_run_id -> process_runs         +-- id
+-- scenario_label                        +-- run_id -> process_runs
+-- started_at, completed_at              +-- stage_name (enum)
+-- created_by (nullable)                 +-- sequence_index
                                          +-- health_score
predictions                               +-- risk_level
+-- id                                    +-- anomaly_flag, anomaly_score
+-- run_id -> process_runs                +-- indicators (JSON)
+-- model_version_id -> model_versions    +-- constraint_breaches (JSON)
+-- label, p_fail, threshold_used         +-- notes (JSON array)
+-- risk_level                            +-- duration_ms
+-- risk_drivers (JSON)                   +-- created_at
+-- explanation (JSON: SHAP top-k)
+-- stage_risk_mass (JSON)                recommendations
+-- caveats (JSON)                        +-- id
+-- latency_ms                            +-- run_id -> process_runs
+-- created_at                            +-- option_rank
                                          +-- changes (JSON: param, from, to)
model_versions                            +-- predicted_p_fail
+-- id                                    +-- predicted_label
+-- name (fusion|thermal_sub|anomaly)     +-- delta_p_fail
+-- semver                                +-- constraint_status
+-- algorithm                             +-- stability_score
+-- trained_at                            +-- extrapolation_flag
+-- dataset_hash                          +-- cycle_time_delta_s
+-- git_sha                               +-- search_method
+-- metrics (JSON)                        +-- created_at
+-- artifact_path
+-- is_active                             simulation_events
                                          +-- id
                                          +-- run_id -> process_runs
                                          +-- event_type
                                          +-- payload (JSON)
                                          +-- created_at
```

17.3 Entities deliberately excluded

Considered	Verdict	Why
`users`, `roles`, `permissions`	Excluded	No auth in scope
`simulation_scenarios` as a separate table	**Merged into `process_runs`** via `is_whatif` + `parent_run_id` + `scenario_label`. A scenario *is* a run. A separate table would duplicate every column	
`process_parameters` as a tall key-value table	**Rejected** in favour of a wide typed `run_parameters` table	The parameter set is fixed and known; EAV destroys type safety, validation, and query ergonomics for zero flexibility benefit
`alerts`, `notifications`	Excluded	No one to notify
`audit_log`	Excluded; `simulation_events` covers what matters	

17.4 Key indexes

`process_runs(created_at)`, `process_runs(machine_id, created_at)`, `process_runs(parent_run_id)`, `stage_results(run_id, sequence_index)`, `predictions(run_id)`, `simulation_events(run_id, created_at)`.

17.5 Dataset storage (separate from the application DB)

```
data/
  raw/          immutable source files + source_manifest.json (URL, licence, date, hash)
  interim/      cleaned, pre-feature-engineering
  processed/    model-ready parquet + the exact split indices used
  synthetic/    generator config + seed + output + generator_version
  reports/      EDA outputs, validation reports, evaluation reports
```

Dataset files are **not** loaded into the application database. The application DB stores *runs*, not the training corpus. Keeping these separate prevents a common student mistake: training from live application tables and creating circular data dependencies.

---

18. Event-Driven / State-Driven Simulation

18.1 Architecture decision

The simulation is a **server-side finite state machine** that emits **events**, streamed to the browser over **Server-Sent Events (SSE)**.

18.2 Transport comparison

Option	Fit	Verdict
Polling (`GET /runs/{id}` every 300 ms)	Works, trivially simple, but wasteful and produces visibly choppy stage transitions	Acceptable fallback; implement as the degradation path
WebSocket	Full duplex, but the only upstream messages are control commands (pause/resume) which are perfectly served by ordinary REST calls. Adds connection lifecycle management, reconnect logic, and a second protocol to test	**Rejected** — duplex is unused
Server-Sent Events	Unidirectional server->client, native `EventSource` API with automatic reconnection, plain HTTP, trivially proxied, natively supported by FastAPI's `StreamingResponse`	**Selected**

Recommendation: SSE for state streaming + REST for all commands. This is the simplest architecture that is genuinely robust, and the asymmetry (server pushes state, client sends commands) exactly matches the problem.

18.3 Event flow

```
   CLIENT                          SERVER
     |                               |
     |--- POST /runs ---------------->|  create run, validate, persist
     |<-- 201 {run_id} ---------------|
     |                               |
     |--- GET /runs/{id}/events ----->|  open SSE stream
     |--- POST /runs/{id}/start ----->|  FSM begins
     |                               |
     |<== event: run.started =========|
     |<== event: stage.started  S1 ===|   UI: node -> ACTIVE, pulse
     |                               |   (module executes, ~4 ms)
     |                               |   (presentation delay applied)
     |<== event: stage.completed S1 ==|   UI: node -> COMPLETED, health badge
     |<== event: stage.started  S2 ===|
     |<== event: anomaly.flagged =====|   UI: purple '!' overlay + timeline row
     |<== event: stage.completed S2 ==|
     |          ...                  |
     |<== event: prediction.ready ====|   UI: result card reveal
     |<== event: run.completed =======|   UI: enable WHY / WHAT-IF / RECOMMEND
     |                               |
     |--- POST /runs/{id}/pause ----->|  FSM pauses (REST command)
     |<== event: run.paused ==========|
```

18.4 Event catalogue

Event	Payload	UI effect
`run.started`	run_id, parameters snapshot	Initialise pipeline
`stage.started`	stage, index, inputs_in_scope	Node -> ACTIVE, populate active-stage panel
`stage.progress`	stage, pct	Progress ring (optional)
`stage.completed`	stage, health, risk, indicators, notes	Node -> COMPLETED with colour + badge
`anomaly.flagged`	scope, score, contributors	Anomaly overlay + timeline entry
`constraint.breached`	rule_id, severity, message	Warning chip; BLOCKED if critical
`stage.blocked`	stage, reason	Halt, show override dialog
`prediction.ready`	full PredictionResult	Result card
`run.completed` / `run.paused` / `run.reset` / `run.aborted`	run_id	Control state

Every event carries `run_id`, a monotonically increasing `seq`, and a timestamp. `seq` lets the client detect gaps after an SSE reconnect and re-sync with a single `GET /runs/{id}`.

18.5 Server-side execution model

- The FSM runs as an `asyncio` task per active run, holding the `SimulationState` in an in-process registry keyed by `run_id`, and persisting each completed stage to the database.
- Presentation delays use `await asyncio.sleep(...)` — non-blocking, so concurrent runs are fine.
- Events are pushed onto an `asyncio.Queue` per run; the SSE endpoint drains that queue.
- **Crash/restart recovery:** on startup, any run left in a RUNNING state is marked `ABORTED` with a reason. The completed stage rows survive, so the run remains inspectable. This is the honest, simple behaviour; resumable durable workflows are out of scope and should be named as such.
- **Stepwise mode** skips the async task entirely: each `POST /advance` executes exactly one stage synchronously and returns its result. This is the mode used for debugging and for the deterministic screenshots in the report.

---

19. Dataset Strategy

19.1 The honest situation

There is no widely available, openly licensed injection-molding dataset that contains **all** of: named physical process parameters, material lot attributes, machine identity, and a binary quality label, at sufficient volume. Candidate public datasets each fail on at least one axis:

Dataset family	What it offers	Why it is insufficient alone
UCI SECOM (semiconductor)	1567 rows, 590 anonymous sensors, binary pass/fail, strongly imbalanced	Anonymous features -> no explainability, no what-if, no recommendations. Wrong domain
Bosch Production Line Performance (Kaggle)	Huge, real, staged production line, binary response	~4200 anonymised features, extreme imbalance, enormous size; anonymity kills the entire value proposition
Multi-stage continuous-flow manufacturing (Kaggle)	Real, named-ish process variables, multi-stage structure	Regression targets, different process, limited parameter overlap
Injection-molding datasets on Kaggle/Mendeley/UCI ML repository	Some contain melt temperature, injection pressure, cycle time, part quality class	Typically small (hundreds to a few thousand rows), single machine, no material-lot attributes, licence and provenance vary
Manufacturing “quality prediction” toy datasets	Clean, balanced, tidy	Usually already synthetic and undocumented — the worst of both worlds

19.2 Recommended strategy: anchored hybrid

A **three-tier dataset strategy**, fully documented, where every row is traceable to its tier:

```
TIER 1  REAL ANCHOR DATASET
  A genuine public injection-molding (or nearest-equivalent process) dataset,
  used to: (a) validate that the pipeline works on real data,
           (b) calibrate the marginal distributions and noise levels of Tier 2,
           (c) provide a real-data evaluation section in the report.
  Every row tagged  source = "real", with the dataset licence recorded.

TIER 2  DOCUMENTED SYNTHETIC GENERATOR (primary training corpus)
  A parameterised generator producing 20,000-50,000 shots across 6-10 virtual
  machines, 3-5 resin grades, and 200+ material lots, with failure labels
  produced by a DOCUMENTED defect-mechanism model (Section 19.4).
  Every row tagged  source = "synthetic", generator_version, seed.

TIER 3  AUGMENTATION (optional, P1)
  Controlled noise injection, sensor-drift simulation, and lot-to-lot variation
  applied to Tier 1 and Tier 2 rows to test robustness.
  Every row tagged  source = "augmented", with its parent row id.
```

19.3 Terminology used strictly

Term	Meaning in this project
Real data	Recorded from physical manufacturing equipment by a third party, obtained under a stated licence
Simulated data	Output of a *physics* solver — **this project produces none**
Synthetic data	Generated by a documented statistical/rule-based generator whose assumptions come from published process-engineering literature; it encodes beliefs about the process, not measurements of it
Augmented data	Real or synthetic rows transformed by noise, jitter, or drift to expand coverage

The report must contain a sentence of exactly this weight: *“The primary training corpus is synthetic. It is generated from documented process-engineering relationships and is intended to exercise and evaluate the system architecture. Model performance figures on synthetic data measure the system’s ability to recover the generator’s structure; they are not evidence of real-world accuracy.”*

This is not a weakness to hide. A project that states it clearly and still delivers rigorous methodology scores higher than one that quietly passes off synthetic data as industrial.

19.4 Synthetic generator design (the part that must not be arbitrary)

Wrong approach: `label = 1 if random() < 0.1`. Or: uniformly random parameters with a random label. This produces a dataset with no learnable structure and no defensible claim.

Correct approach — a mechanism-based generator:

```
STEP 1  SAMPLE CONTEXT
  machine   ~ categorical over the machine fleet, each with a persistent bias
              vector (offsets on temperature and pressure delivery, plus a
              drift term increasing with days_since_maintenance)
  product   ~ categorical, carrying wall_thickness, projected_area, cavity_count
  material  ~ resin grade -> MFI ~ Normal(grade_mean, grade_sd), truncated
              moisture ~ LogNormal (mostly low, occasional high lot)
              regrind  ~ Beta scaled to [0, 0.30]

STEP 2  SAMPLE A RECIPE
  Operators do not set parameters uniformly at random. They set them near a
  grade-specific and product-specific nominal recipe, with excursions.
      param = nominal(grade, product) + Normal(0, sd) + occasional_excursion
  Excursions (5-12% of shots) are drawn from a heavier-tailed distribution and
  represent setup errors, rushed cycle-time reduction, and material changeovers.
  Parameter CORRELATIONS are imposed deliberately: an operator raising injection
  velocity often also raises melt temperature; a rushed cycle shortens both
  cooling and hold time together.

STEP 3  COMPUTE DEFECT MECHANISM SCORES (documented formulas)
  short_shot   = s( a1*(viscosity_proxy) - a2*fill_index )
  flash        = s( b1*(cavity_pressure_est - clamp_capacity) + b2*over_pack )
  sink_void    = s( c1*(required_packing(thickness) - packing_integral) )
  warp         = s( d1*(1 - cooling_adequacy) + d2*residual_stress_proxy
                    + d3*mold_temp_imbalance )
  splay        = s( e1*moisture_excess * melt_temp_excess )
  degradation  = s( f1*thermal_exposure )
  where s() is a logistic squashing function and every coefficient is chosen so
  that the resulting marginal defect rates match published/typical plant ranges.

STEP 4  COMBINE INTO A LABEL
  p_defect = 1 - PRODUCT over mechanisms of (1 - p_mechanism)
  label    = Bernoulli(p_defect)            # stochastic, NOT deterministic
  defect_mode = argmax mechanism, recorded as a secondary column

STEP 5  ADD REALISM
  - measurement noise on every recorded sensor value (recorded != setpoint)
  - 1-3% missing values, with a realistic missingness pattern (some sensors
    fail more often on older machines)
  - temporal ordering with slow drift per machine
  - class imbalance tuned to a realistic 6-12% failure rate
  - a small fraction of label noise (inspection error), e.g. 1-2%

STEP 6  DOCUMENT
  Persist generator_config.yaml, the random seed, the coefficient table, and a
  written justification with literature references for each nominal window.
```

Why this design matters technically: the generator deliberately creates **non-monotone** effects (pressure too low -> short shot; too high -> flash) and **interactions** (moisture only matters at high temperature). A logistic regression will therefore underperform, tree ensembles will do well, and SHAP dependence plots will show genuine non-linear structure. The dataset is hard in the ways the project claims to address.

The critical caution: the fusion model is being trained on data generated by known mechanisms. Achieving high accuracy proves the pipeline recovers the generator — nothing more. The report must not present synthetic-data metrics as industrial accuracy. Report Tier-1 real-data metrics separately and prominently, however modest they are.

19.5 Required columns

Group	Columns
Identity	`shot_id`, `batch_id`, `timestamp`, `machine_id`, `product_id`, `material_lot_id`
Material	`resin_grade`, `mfi`, `moisture_pct`, `regrind_pct`, `colourant_pct`, `drying_time_h`, `dryer_temp_c`
Thermal	`barrel_temp_rear/mid/front`, `nozzle_temp_c`, `melt_temp_c`, `screw_rpm`, `back_pressure_bar`, `residence_time_s`
Injection	`inj_velocity_mm_s`, `inj_pressure_bar`, `fill_time_s`, `shot_size_mm`, `switchover_position_mm`, `clamp_force_ton`
Packing	`hold_pressure_bar`, `hold_time_s`, `cushion_mm`
Cooling	`mold_temp_c`, `cooling_time_s`, `cycle_time_s`, `ejection_force_proxy`
Product context	`wall_thickness_mm`, `projected_area_cm2`, `cavity_count`, `nominal_shot_weight_g`
Machine context	`machine_age_days`, `days_since_maintenance`, `max_inj_pressure_bar`, `max_clamp_force_ton`
Target	`quality_pass` (0/1)
Secondary (synthetic only)	`defect_mode`, `p_defect_true`, `source_tier`, `generator_version`

`p_defect_true` and `defect_mode` are **never** used as model inputs. They exist purely for evaluation diagnostics (e.g. checking calibration against the true generating probability) and must be excluded by the feature schema, with a unit test asserting their absence.

19.6 Size and balance targets

- 25,000–40,000 rows: large enough for GBDT and a modest MLP, small enough to train in under a minute on a laptop.
- 6–10 machines, so machine-wise splits and machine-effect analysis are meaningful.
- 200+ material lots, so lot-level grouping is possible.
- Failure rate 6–12%: realistic imbalance, still ~2,000–4,000 positive cases.
- Chronologically ordered with a usable `timestamp` for temporal splits and SPC.

---

20. Technology Stack (Final, with justification)

Layer	Technology	Used by	Mandatory?	Complexity	Justification	Alternative
Language (backend)	Python 3.11+	Everything server-side	Mandatory	Low	The ML ecosystem lives here; one language for pipeline and API	—
API framework	FastAPI	API layer	Mandatory	Low	Pydantic validation is load-bearing for the constraint registry; free OpenAPI schema	Flask (no async, weaker validation)
ASGI server	Uvicorn	Runtime	Mandatory	Low	Standard	Hypercorn
Validation	Pydantic v2	Schemas, constraints	Mandatory	Low	Ranges and cross-field validators map 1:1 to the domain	Manual checks
ORM	SQLAlchemy 2.0	Data layer	Mandatory	Medium	DB portability SQLite -> Postgres without rewrites	Raw SQL
Migrations	Alembic	Data layer	Recommended	Low	Schema will change weekly during the project	Drop-and-recreate
DB (dev)	SQLite	Data layer	Mandatory	Low	Zero setup, file-based, CI-friendly	—
DB (demo)	PostgreSQL	Data layer	Optional	Medium	Professional deployment story	Stay on SQLite
ML core	scikit-learn	Pipeline, baselines, calibration, IsolationForest	Mandatory	Low	The pipeline/ColumnTransformer abstraction is what prevents train-serve skew	—
Gradient boosting	XGBoost (primary), LightGBM (comparison)	Fusion model, stage sub-models	Mandatory	Low	Best tabular performance; fast TreeSHAP	CatBoost
Deep learning	PyTorch	MLP benchmark, autoencoder anomaly detector	Optional (P1)	Medium	Only where DL is justified (Sec 9.5, 12.2)	TensorFlow/Keras
Explainability	SHAP	Explain service	Mandatory	Medium	TreeExplainer is exact and fast for tree models	Permutation importance only
Imbalance	imbalanced-learn	Training (if needed)	Optional	Low	Pipeline-compatible SMOTE variants	Class weights alone
Tuning	Optuna	Offline training	Recommended	Low	Better than grid search per unit of compute; easy study persistence	RandomizedSearchCV
Optimization	SciPy `differential_evolution`	Recommendation refinement	Optional (P1)	Medium	Derivative-free, handles bounds natively	Coordinate search alone
Experiment tracking	MLflow (local file backend)	Offline training	Recommended	Medium	Model registry + metrics history; huge report value for one day of work	CSV log + manual notes
Serialization	joblib	Model store	Mandatory	Low	Standard for sklearn objects	ONNX (unnecessary)
Data	pandas, numpy, pyarrow	Pipeline	Mandatory	Low	—	polars (faster, less familiar)
Testing	pytest, httpx	All	Mandatory	Low	—	unittest
Linting	ruff + black + mypy (partial)	All	Recommended	Low	Cheap professional polish	—
Frontend language	TypeScript	Frontend	Mandatory	Medium	Types derived from the OpenAPI schema catch contract drift at compile time	JavaScript
Frontend framework	React 18 + Vite	Frontend	Mandatory	Medium	Ecosystem and team familiarity; Vite is fast	Vue, Svelte
Styling	Tailwind CSS	Frontend	Mandatory	Low	Fast, consistent industrial look	CSS modules
Components	shadcn/ui (Radix)	Frontend	Recommended	Low	Accessible sliders, dialogs, tabs, tooltips	Hand-rolled
Charts	Recharts	Analytics, SHAP, sweeps	Mandatory	Low	Declarative React charting, covers every chart needed	ECharts, Plotly
Process graphic	Hand-built SVG + Framer Motion	Stage pipeline	Mandatory	Medium	Fixed 6-node chain; a graph library is overkill	React Flow
Server state	TanStack Query	Frontend	Recommended	Low	Caching and retries for run/history data	useEffect + fetch
Client state	Zustand	Frontend	Recommended	Low	Small store for simulation UI state	Context, Redux
Live updates	SSE (`EventSource`)	Frontend + API	Mandatory	Low	Matches the unidirectional data flow exactly	WebSocket, polling
Containerisation	Docker + Compose	Deployment	Optional (P2)	Medium	Reproducible demo; one command to run everything	Local processes
CI	GitHub Actions	Repo	Optional (P2)	Low	Runs lint + tests on push; cheap credibility	Manual

Explicitly rejected technologies and why: Kubernetes (no scale), Celery + Redis (no long jobs, no broker justification), Kafka (no streams), Three.js (cost >> value), Next.js (SSR unused), GraphQL (REST is simpler for a fixed, small surface), microservices (deployment pain, zero benefit), Airflow (one training script), Redis cache (an in-process LRU is sufficient), LangChain/agent

frameworks (no agentic problem exists — see Section 8.1).

---

21. Development Workflow

21.1 Repository structure

```
smart-mfg-quality/
  backend/            FastAPI application (Section 16.2)
  frontend/           React + TS SPA
  ml_pipeline/
    01_ingest/  02_eda/  03_features/  04_train/  05_evaluate/
    synthetic/        generator + config + seeds
    notebooks/        exploratory only; nothing production depends on a notebook
  data/               raw / interim / processed / synthetic / reports (gitignored)
  models/             versioned bundles (gitignored; hashes committed)
  docs/
    architecture.md  decisions/ADR-*.md  api.md  dataset_card.md  model_card.md
  tests/
  docker/
  .github/workflows/
```

21.2 Practices

- **Branching:** trunk-based with short feature branches; PR-style review between team members even if self-merged.
- **Architecture Decision Records:** one short ADR per significant choice (SSE over WebSocket; GBDT over DL; synthetic-primary dataset; no agents; no auth). These are directly reusable as report sections and are excellent viva material.
- **Definition of done** for any module: implemented, unit-tested, documented in `docs/`, integrated behind its interface, and demonstrated end-to-end.
- **Reproducibility:** every training run records git SHA, dataset hash, seed, and full config. A `make train` target reproduces a model bundle from scratch.
- **Contract-first:** the OpenAPI schema is generated from FastAPI and typegen'd into the frontend. The frontend never hand-writes an API type.
- **Weekly integration checkpoint:** the system must run end-to-end (even trivially) from Phase 10 onward, every week. Never allow a long integration-free stretch.

21.3 Team split (assuming 3–4 members)

Role	Owns
ML engineer	Dataset strategy, generator, pipeline, models, tuning, evaluation, SHAP, anomaly detector
Backend engineer	FastAPI, simulation engine, FSM, SSE, database, inference service, recommendation service
Frontend engineer	React SPA, stage visualization, what-if workbench, charts, analytics
Integrator / domain lead	Constraint registry, remedy tables, domain justification, documentation, testing, demo script

The constraint registry is the shared interface between all four roles. Whoever owns it owns the project's domain credibility.

---

22. Implementation Phases

Difficulty: L = low, M = medium, H = high. Each phase states its gate — what must not begin before it completes.

Phase 0 — Requirements & domain finalization *(difficulty L)*

- **Objectives:** lock the domain, the product, the parameter list, and the constraint registry v1.
- **Tasks:** literature survey of injection-molding parameter windows and defect mechanisms; write `constraints.yaml` and `remedies.yaml` with citations; finalise the parameter schema; write the four-claim statement (Section 1.4).
- **Deliverables:** domain brief, constraint registry v1, parameter dictionary, ADR-001 (domain selection).
- **Dependencies:** none.
- **Acceptance:** every parameter has a documented nominal, min, max, step, unit, actionable flag, and change cost.
- **Gate:** no dataset generation and no modelling before the registry exists — the generator depends on it.

Phase 1 — Dataset & data pipeline *(difficulty M)*

- **Objectives:** acquire the Tier-1 anchor dataset; build and validate the Tier-2 generator.
- **Tasks:** source and licence-check public data; implement the mechanism-based generator; write the data contract and validation suite; produce the dataset card.
- **Deliverables:** `data/raw` with manifest, generator + config + seed, generated corpus, dataset card, validation report.
- **Dependencies:** Phase 0.
- **Acceptance:** marginal distributions and failure rate match the documented targets; validation suite passes; generator is deterministic given its seed.
- **Gate:** no model training until validation passes and the split protocol is written down.

Phase 2 — EDA & preprocessing *(difficulty L–M)*

- **Objectives:** understand the data; build the leak-free preprocessing pipeline.
- **Tasks:** univariate/bivariate/target-conditioned EDA; correlation and VIF; implement the ColumnTransformer; implement grouped + temporal splitting; write the leakage checklist.
- **Deliverables:** EDA report, preprocessing pipeline object, split module, leakage checklist.
- **Acceptance:** splits verified group-disjoint and time-ordered; a deliberate leakage test (train on shuffled target) yields chance performance.
- **Gate:** no model selection before the split protocol is fixed.

Phase 3 — Baseline ML *(difficulty L)*

- **Objectives:** establish honest baselines.
- **Tasks:** majority-class baseline, logistic regression, decision tree, random forest; full metric suite; confusion matrices; threshold analysis.
- **Deliverables:** baseline evaluation report, metric harness reused by every later model.
- **Acceptance:** baselines beat the majority-class baseline on PR-AUC; metrics reproducible from a single command.
- **Gate:** no advanced modelling until the metric harness is trusted.

Phase 4 — Advanced ML / DL *(difficulty M)*

- **Objectives:** select and tune the fusion model; benchmark DL fairly.
- **Tasks:** XGBoost/LightGBM tuning via Optuna; MLP with embeddings under the identical split; calibration; model card; MLflow logging.
- **Deliverables:** tuned model bundle v1, calibration report, GBDT-vs-DL comparison section.
- **Acceptance:** PR-AUC and FAIL-recall improvement over baselines with fold-level confidence intervals; calibration Brier score improved; every claim reproducible.
- **Gate:** stage modules may be developed in parallel, but the fusion model is not finalised before its inputs are.

Phase 5 — Stage intelligence modules *(difficulty M)*

- **Objectives:** implement the five modules as pure, tested functions.
- **Tasks:** shared base interface; per-stage rules, indicators, aggregation; upstream-context contract; out-of-fold training for any stage sub-model; unit tests including purity and determinism tests.
- **Deliverables:** `intelligence/` package, stage specification documents, test suite.
- **Acceptance:** every module is deterministic, side-effect-free, and produces the documented output contract; golden-case tests for each documented defect mechanism produce the expected risk direction.
- **Gate:** the fusion feature set cannot be frozen before this completes.

Phase 6 — Fusion quality engine *(difficulty M)*

- **Objectives:** combine raw + stage + engineered + context; retrain; verify no leakage from stage sub-models.
- **Tasks:** feature builder shared by training and inference; stacking-safe out-of-fold stage outputs; ablation study (raw-only vs stage-only vs fusion).
- **Deliverables:** fusion model bundle, ablation report.
- **Acceptance:** the ablation table exists and is discussed honestly, including the case where fusion does *not* beat raw-only (a real and publishable outcome).
- **Gate:** no explainability work before the feature set is frozen.

Phase 7 — Explainability & anomaly detection *(difficulty M)*

- **Objectives:** local + global explanations; the three anomaly levels.
- **Tasks:** SHAP service with cached background; stage-risk-mass computation; Isolation Forest; optional autoencoder; SPC rules; caveat/out-of-distribution detection.
- **Deliverables:** explain service, anomaly module, global explanation report, calibration and reliability plots.
- **Acceptance:** local explanation returns in under ~200 ms; attribution sanity-checked against permutation importance; anomaly detector flags injected synthetic anomalies at the target rate.
- **Gate:** the recommendation engine must not be built before the feasibility signals (anomaly score, OOD detection) exist — it depends on them.

Phase 8 — Recommendation engine *(difficulty H)*

- **Objectives:** the three-stage funnel with a hard feasibility guard.
- **Tasks:** remedy table; candidate proposal; coordinate/local search; feasibility guard (all six checks); ranking with change cost; optional differential-evolution refinement; evaluation harness.
- **Deliverables:** recommendation service, remedy table, recommendation evaluation report.
- **Acceptance:** 100% of returned options satisfy hard constraints (asserted by test); reported success rate on a held-out set of failing recipes; reported stability rate; reported extrapolation-rejection rate.
- **Gate:** do not expose recommendations in the UI until the guard is tested. An unguarded recommendation is the single highest-risk output in the whole project.

Phase 9 — What-if simulation engine *(difficulty M)*

- **Objectives:** dependency-graph partial re-simulation, caching, diffs, sensitivity.

- **Tasks:** dependency graph; invalidation logic; LRU cache; run lineage; sweep endpoint; flip-point detection; A/B diff.
- **Deliverables:** what-if service, sensitivity endpoint, diff contract.
- **Acceptance:** partial and full re-simulation produce byte-identical results (property test); a single-parameter what-if completes in under 100 ms end-to-end.
- **Gate:** none downstream; but do not start the frontend what-if workbench before the diff contract is frozen.

Phase 10 — Backend / API *(difficulty M)*

- **Objectives:** the full REST surface, the FSM, and SSE.
- **Tasks:** routers, schemas, error envelope, FSM, event bus, SSE endpoint, database models, Alembic migrations, repositories, logging, rate limits.
- **Deliverables:** running API with OpenAPI docs, migration history, API test suite.
- **Acceptance:** every documented endpoint implemented and tested; SSE verified with a scripted client; OpenAPI schema generates valid frontend types.
- **Gate:** frontend integration begins only after the schema is stable (a mock server may be used before that).

Phase 11 — Professional frontend *(difficulty H)*

- **Objectives:** all seven screens, with the process-control workspace as the centrepiece.
- **Tasks:** layout and design system; SVG stage pipeline with animated states; SSE client with reconnect and `seq` gap handling; parameter panel with registry-driven sliders; result card; explainability panel; what-if workbench; recommendation cards; history and analytics; model info.
- **Deliverables:** complete SPA.
- **Acceptance:** a full run is watchable end-to-end without a page refresh; every backend state is visually represented; the UI degrades gracefully to polling if SSE fails.
- **Gate:** do not begin polish/animation refinement before all screens are functionally complete.

Phase 12 — Integration *(difficulty M)*

- **Objectives:** one coherent system.
- **Tasks:** wire every path end-to-end; unify error handling; loading and empty states; seed data for demo machines, products, and lots; performance pass.
- **Deliverables:** integrated build, seed script.
- **Acceptance:** the full user journey (Section 5) executes without manual intervention or console errors.

Phase 13 — Testing *(difficulty M)*

- **Objectives:** demonstrable engineering quality.
- **Tasks:** unit tests (modules, features, risk engine, constraints), API integration tests, an end-to-end journey test, ML regression tests (metrics must not drop below a recorded floor), recommendation constraint-compliance property tests, load sanity test.
- **Deliverables:** test suite, coverage report, CI pipeline.
- **Acceptance:** CI green; critical paths covered; a documented manual test script for the demo.

Phase 14 — Deployment *(difficulty L–M)*

- **Objectives:** a reproducible demo environment.
- **Tasks:** Dockerfiles, Compose (api + db + static frontend), environment configuration, model artifact mounting, seed-on-first-run, README quickstart.
- **Deliverables:** `docker compose up` brings the whole system live.
- **Acceptance:** a clean machine runs the system from the README in under 10 minutes.

Phase 15 — Documentation & presentation *(difficulty L)*

- **Objectives:** the academic deliverables.
- **Tasks:** final report, architecture diagrams, dataset card, model card, ML evaluation report, limitations chapter, ADR compilation, demo script, slide deck, recorded backup demo video.
- **Deliverables:** everything in Section 25.
- **Acceptance:** the limitations chapter explicitly contains the four-claim distinction; the demo runs in under 8 minutes with a rehearsed failure-recovery path.

22.1 Suggested schedule (12 weeks)

Week	Focus
1	Phase 0, start Phase 1
2	Phase 1 complete, Phase 2
3	Phase 3, start Phase 10 skeleton (API scaffold in parallel)
4	Phase 4
5	Phase 5
6	Phase 6, start Phase 11 scaffold
7	Phase 7
8	Phase 8
9	Phase 9, Phase 10 complete
10	Phase 11
11	Phase 12, Phase 13
12	Phase 14, Phase 15, rehearsal

Parallelisation note: the frontend scaffold and API scaffold should start early against mocked contracts. The single biggest schedule risk in projects of this shape is leaving integration until the final fortnight.

---

23. MVP vs Enhanced vs Final Showcase

23.1 MVP — the minimum that demonstrates the concept

Target: end of Week 9. If everything after this fails, the project is still a pass-worthy, coherent system.

- Synthetic dataset (Tier 2) + validated preprocessing pipeline.
- XGBoost fusion model, calibrated, with honest metrics.
- Five stage intelligence modules, rule + indicator based.
- Sequential simulation with a real FSM, stepwise mode.
- FastAPI backend with run, predict, what-if, and explain endpoints; SQLite.
- React frontend: stage pipeline visualization, parameter panel, result card, SHAP explanation panel, single-parameter what-if.
- Basic recommendation: rule-guided proposals + coordinate search + feasibility guard.
- Run history persisted.

23.2 Enhanced version

- SSE live event streaming with animated stage progression, pause/resume/step, speed control.
- Full three-stage recommendation funnel including differential-evolution refinement, ranked options with change cost and cycle-time impact.
- Isolation Forest global anomaly detection + per-stage anomaly badges + SPC drift.
- Sensitivity sweeps with flip-point markers; A/B scenario comparison.
- Analytics dashboard with failure patterns by machine, lot, and parameter bucket.
- MLflow experiment tracking and a model-info screen.
- Tier-1 real-dataset validation section.
- Comprehensive test suite + CI.

23.3 Final showcase version

Everything above, plus the elements that make the demo memorable:

- **Autoencoder anomaly detector** — the justified deep-learning component, with per-feature reconstruction error localising the anomaly.
- **2-D response surface heatmap** for any two parameters, with the current operating point and the decision boundary drawn on it. This single visual explains the entire project in one image and is the strongest slide in the deck.
- **BLOCKED-stage demonstration** — deliberately enter a degradation-ceiling breach and show the system refusing to proceed. It proves domain constraints outrank model output.
- **Recommendation stability panel** — show the perturbation test and the extrapolation rejection in action.
- **Run lineage tree** — visualise the parent run and its what-if branches.
- **Docker Compose one-command deployment.**

23.4 Features that must NOT be built

Restating Section 4.1 P3 as an explicit prohibition: no auth/RBAC, no real-time hardware ingestion, no 3D rendering, no reinforcement learning, no microservices, no message broker, no physics solver, no mobile app, no multi-tenant support, no LLM chatbot layer over the results. Each would consume a week and add nothing to the evaluation criteria.

---

24. Testing & Evaluation

24.1 ML evaluation

Metric	Role
PR-AUC	Primary selection metric (imbalanced positive class)
Recall on FAIL	Co-primary — a missed defective part is the costly error
Precision on FAIL	Reported; governs alert fatigue
F1 / F-beta (beta=2)	Reported, beta>1 to reflect the asymmetric cost
ROC-AUC	Reported for comparability, never for selection under imbalance
Accuracy	Reported only alongside the majority-class baseline, to show it is uninformative
Confusion matrix	At the tuned threshold and at 0.5, both shown
Brier score + reliability diagram	Calibration quality, before and after
Per-machine and per-resin-grade breakdown	Fairness/robustness across contexts
Learning curve	Does more data help? Informs the dataset-size claim

False negatives are the headline risk. A FAIL predicted as PASS ships a defective part. The decision threshold is therefore tuned on the validation PR curve to a documented operating point (e.g. minimum acceptable FAIL-recall of 0.85), and the resulting precision cost is stated explicitly. The threshold is a *product decision*, recorded in the model bundle, not a silent 0.5.

24.2 Stage-level evaluation

Target	Method
Stage risk vs outcome	Failure rate conditioned on each stage's risk band — does HIGH actually fail more? Report as a monotonicity check with confidence intervals
Stage sub-model quality	Standard classification metrics on their own out-of-fold predictions
Anomaly detector	Inject known synthetic anomalies (parameter combinations outside the generator's normal region) and measure detection rate + false-alarm rate on normal data
Rule coverage	Fraction of runs triggering each rule; a rule that never fires is dead weight, one that always fires is mis-specified
Stage contribution	Ablation: remove one stage's outputs from fusion and measure the metric change

24.3 Recommendation evaluation

Evaluated on a held-out set of ~500 failing recipes:

Metric	Definition	Target
Flip rate	% of failing recipes where at least one option flips the predicted label	Report (not a target to game)
Mean predicted improvement	Mean reduction in `p_fail`	Report with a distribution, not just a mean
Constraint satisfaction	% of returned options passing all hard constraints	**Must be 100%** — asserted by test
Extrapolation rate	% of options flagged out-of-distribution	Report; lower is better
Stability rate	% of options whose label survives 20 small perturbations	Report; target >90% for the top option
Minimality	Mean number of parameters changed	Lower is better for operator acceptance
Cycle-time cost	Mean change in cycle time	Report — improvements that cost 30% cycle time are not free
Generator ground-truth check (synthetic only)	Does the recommended recipe reduce the *generator's* true `p_defect`, not just the model's estimate?	**The most important diagnostic in the project**

That last row deserves emphasis. Because the synthetic generator's true mechanism is known, the project can measure whether the recommendation engine is genuinely improving the underlying process or merely exploiting the surrogate model. This is a rigorous, self-contained validation that most student projects cannot perform — and it directly addresses the "model exploitation" risk. It must be a headline result in the report, with the honest caveat that it validates against the generator's assumptions, not against physical reality.

24.4 System evaluation

Aspect	Target	Method
Single prediction latency (p95)	< 150 ms	API benchmark, 1000 requests
SHAP explanation latency (p95)	< 300 ms	Benchmark with the cached background
Single what-if (partial re-sim)	< 100 ms	Benchmark
Sensitivity sweep (40 points)	< 1.5 s	Benchmark
Recommendation (full funnel)	< 5 s	Benchmark; must show a progress indicator beyond 1 s
Full simulated run (instant mode)	< 300 ms	Benchmark
SSE event delivery jitter	< 100 ms	Scripted client measurement
Frontend interaction responsiveness	60 fps during stage animation	DevTools profiling
Concurrent runs	10 simultaneous without degradation	Load script
Error handling	No unhandled exception reaches the user; every error has a code and a hint	Fault-injection tests
Reliability	Server restart mid-run leaves inspectable partial state	Kill test

24.5 Test inventory

- **Unit:** stage modules (including golden defect-mechanism cases), feature builder, risk engine, constraint loader, feasibility guard, cache keys, FSM transitions.
- **Property:** stage purity (same input -> same output), partial re-sim equals full re-sim, every recommendation satisfies constraints.
- **Integration:** each API endpoint, error envelopes, SSE stream, database persistence.
- **End-to-end:** the complete Section 5 journey, scripted.
- **ML regression:** stored metric floors; CI fails if a retrained model drops below them.
- **Leakage tests:** shuffled-target training must yield chance performance; `p_defect_true` and `defect_mode` must be absent from the feature schema.

---

25. Risks and Limitations

25.1 Data and modelling risks

#	Risk	Impact	Likelihood	Mitigation
R1	**Synthetic-data bias** — the model learns the generator, not manufacturing	High	**Certain**	State it prominently; validate on the Tier-1 real dataset; report both separately; ground generator coefficients in literature; never claim industrial accuracy
R2	**Insufficient real data**	Medium	High	Tier-1 dataset used for validation and distribution calibration rather than as the primary training corpus; scope claims accordingly
R3	**Class imbalance** distorts metrics	Medium	Certain	PR-AUC + FAIL-recall selection; tuned threshold; class weights; imbalance ablation
R4	**Data leakage** via random splits, pre-split scaling/imputation, or stage sub-models trained in-fold	**Critical**	Medium	GroupKFold by batch + temporal holdout; all preprocessing inside the pipeline; out-of-fold stacking for stage sub-models; automated leakage tests
R5	**Overfitting** to the synthetic structure	Medium	Medium	Cross-validated tuning, early stopping, learning curves, held-out temporal test, reported fold variance
R6	**Target leakage** from `p_defect_true` / `defect_mode`	Critical	Low	Feature-schema allowlist + unit test asserting exclusion
R7	**Miscalibrated probabilities** driving risk bands and recommendation deltas	High	Medium	Mandatory isotonic/Platt calibration; reliability diagram; Brier score reported before and after
R8	**Correlated features distort SHAP**	Medium	High	Grouped attribution for headline claims; cross-check with permutation importance; document the limitation in the UI

25.2 System and recommendation risks

#	Risk	Impact	Likelihood	Mitigation
R9	**Recommendation engine exploits the model** — finds flat/extrapolated regions rather than better processes	**Critical**	**High**	The six-check feasibility guard; in-distribution enforcement; perturbation stability test; generator ground-truth validation (24.3); never surface an option that fails the guard
R10	**Unrealistic or unsafe suggestions**	High	Medium	Hard constraint filters over rules, not penalties; step-size limits; locked non-actionable parameters; BLOCKED state for critical breaches
R11	**Simulation mistaken for physical simulation** by readers, examiners, or users	High	**High**	Permanent UI disclaimer; Section 7.1 in the report; explicit vocabulary discipline; never use words like “solves”, “physics”, or “digital twin”
R12	**SHAP presented as causal proof**	High	High	Permanent footnote in the panel; a dedicated limitations subsection; explicit association-vs-causation statement
R13	**Arbitrary risk thresholds** with no basis	Medium	High	Derive thresholds from data (empirical failure rates per probability decile) rather than invention; document every threshold’s provenance; keep them in the versioned registry
R14	**Frontend/backend integration complexity**	High	High	Contract-first OpenAPI + generated types; mock server from week 3; weekly integration checkpoints from Phase 10
R15	**Scope creep**	**High**	**Very high**	The P3 prohibition list is binding; any new feature must displace an existing one; ADR required for any stack addition
R16	**Over-engineering** (microservices, brokers, agent frameworks)	High	Medium	Explicit rejection list in Section 20 with written justification; “simplest robust option” as a standing rule
R17	**SSE connection instability** during the demo	Medium	Medium	`seq`-based gap detection + full state re-sync; automatic polling fallback; stepwise mode as a manual backup; recorded demo video
R18	**Model-serving skew** (training features != inference features)	High	Medium	One shared feature builder; feature order read from the persisted schema; an integration test comparing training-time and serving-time vectors for identical inputs
R19	**Team bottleneck** on one member	Medium	Medium	Interface-first development; mocked dependencies; the constraint registry as the shared contract
R20	**Demo failure** on presentation day	High	Low	Seeded deterministic demo run; instant mode; offline build; recorded video; rehearsed 8-minute script

25.3 Fundamental limitations (to be stated, not mitigated)

These cannot be engineered away within this project’s scope, and the report must own them:

1. **No physical validation.** No recommendation has been trialled on a machine. Every improvement figure is model-relative.
2. **Correlation, not causation.** The model learns associations in the (largely synthetic) data. It cannot establish that changing a parameter causes a quality change.
3. **No physics.** No material behaviour, flow, or thermal field is computed. Heuristics such as gate-freeze and cooling-time estimates are explicitly labelled heuristics.
4. **Distribution-bound.** The system is valid only near the data it was fitted on. Novel resins, geometries, or machines are outside its competence, and the OOD flag exists to say so.
5. **Static models.** No online learning, no feedback loop from actual inspection results into retraining.
6. **Single-process scope.** Nothing here transfers to a different manufacturing process without redoing the domain work.
7. **Inspection label granularity.** A binary PASS/FAIL discards defect severity and multi-defect co-occurrence information.

A limitations chapter written at this level of specificity is, in practice, one of the strongest signals of technical maturity a minor project can send.

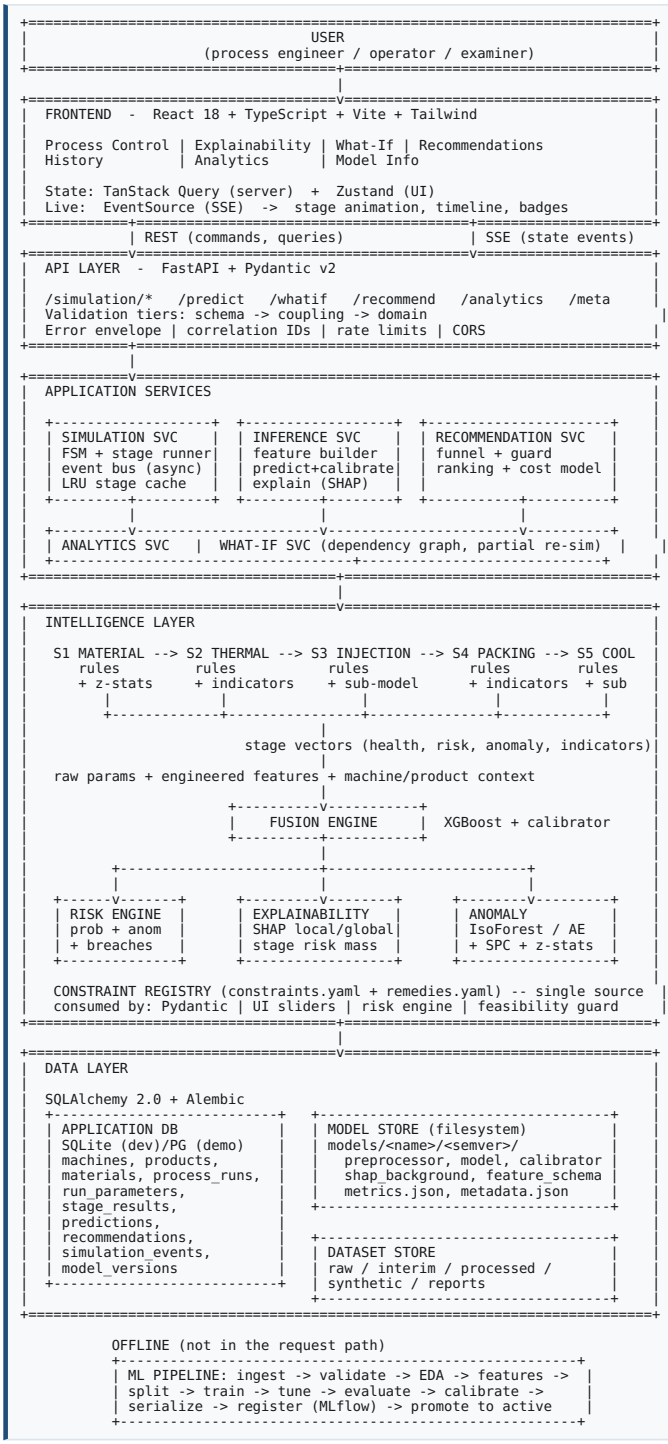
---

26. Final Deliverables

#	Deliverable	Description	Acceptance
D1	Working software	Integrated full-stack system, runnable via Docker Compose	Full journey executes end-to-end
D2	Trained models	Versioned fusion model bundle, stage sub-models, anomaly detector, calibrator	Reproducible from `make train` with a recorded seed
D3	Dataset	Tier-1 anchor (with licence) + Tier-2 generator, config, seed, and corpus	Regenerable byte-identically from config + seed
D4	Dataset card	Provenance, tiers, generator assumptions, limitations	Written, reviewed
D5	Preprocessing pipeline	Serialized, leak-free, shared by training and serving	Skew test passes
D6	Stage intelligence modules	Five documented, tested, pure modules	Unit + golden tests pass
D7	Fusion quality engine	Calibrated classifier + risk engine	Metrics report attached
D8	Explainability system	SHAP local + global, stage risk mass, caveats	Latency target met
D9	Anomaly detection	Per-stage, global Isolation Forest, optional autoencoder, SPC	Detection-rate report
D10	Recommendation engine	Three-stage funnel + feasibility guard	100% constraint compliance test
D11	What-if simulation	Dependency-graph partial re-simulation, sweeps, A/B compare	Equivalence property test passes
D12	Interactive visualization	Process-control workspace with animated stage pipeline	Demonstrable live
D13	Backend API	Documented REST + SSE, OpenAPI schema	All endpoints tested
D14	Database	Schema + migrations + seed data	Migrations run clean from empty
D15	Frontend	Seven-screen SPA	All screens functional
D16	Test suite	Unit, property, integration, e2e, ML regression, leakage	CI green
D17	Architecture documentation	Diagrams + ADRs + module specs	Matches implementation
D18	ML evaluation report	All metrics, ablations, calibration, comparisons, learning curves	Reproducible
D19	Model card	Intended use, training data, metrics, limitations, ethical notes	Written
D20	Final project report	The full academic document including the limitations chapter	Submitted
D21	Presentation deck	~15 slides, architecture + results + live demo + limitations	Rehearsed
D22	Demo script + video	8-minute deterministic walkthrough, plus a recorded backup	Recorded before demo day

27. Final Architecture Diagram

27.1 Complete system view



27.2 Layer responsibilities

Layer	Owns	Must not
Frontend	Presentation, interaction, draft parameter state, animation	Contain business rules or parameter limits (it reads them from `/constraints`)
API	Transport, validation, error shaping, orchestration	Contain ML code or domain logic
Application services	Workflow, state machine, caching, composition	Touch the database directly from inside stage modules
Intelligence	Domain rules, indicators, models, risk, explanation	Perform I/O or hold mutable state
Data	Persistence, migrations, artifact storage	Contain business logic
Offline pipeline	Training, tuning, evaluation, registration	Be invoked from a request handler

28. Final End-to-End Technical Workflow

The complete trace of one interaction, from click to rendered recommendation.

```
[1]  USER selects machine MC-03, product PP-JBOX-42, lot A-771, enters recipe.
```

```
     Frontend builds the sliders from GET /api/v1/constraints, so no value
     outside the admissible window can even be entered.

[2]  POST /api/v1/simulation/runs
     -> Pydantic schema validation (types, ranges from the registry)
     -> coupling validators (hold_pressure <= inj_pressure, melt >= mold + delta)
     -> persist process_runs + run_parameters
     -> return run_id, status = INITIALIZED

[3]  Frontend opens GET /api/v1/simulation/runs/{id}/events  (SSE)
     Frontend sends POST /simulation/runs/{id}/start  (mode = auto, speed = 1x)

[4]  FSM spawns an asyncio task.
     emit run.started

[5]  STAGE S1 MATERIAL_CONDITIONING
     - stage_runner selects the material input slice
     - cache lookup on hash(stage, inputs, upstream, module_version, constraints_version)
     - MISS -> material module executes: spec rules, z-scores, dryness margin,
       viscosity_class
     - result: health 0.86, risk LOW, anomaly false
     - persist stage_results row; emit stage.started / stage.completed
     - publish {viscosity_class, moisture_flag} into upstream context

[6]  STAGE S2 PLASTICIZING/THERMAL
     - consumes thermal slice + upstream context
     - computes thermal_exposure, shear_heat_proxy, effective_melt_temp,
       profile monotonicity
     - degradation exposure high -> risk MEDIUM, anomaly true
     - emit anomaly.flagged + stage.completed
     - publish {effective_melt_temp, viscosity_proxy}

[7]  STAGE S3 INJECTION
     - fill_index = (pressure x velocity) / viscosity_proxy
     - clamp-force guard evaluated; pressure headroom checked
     - short-shot / flash sub-model scored (out-of-fold trained)
     - publish {fill_completion_proxy, cavity_pressure_estimate}

[8]  STAGE S4 PACKING
     - packing_integral, gate_freeze_estimate (labelled heuristic), cushion checks
     - publish {residual_stress_proxy}

[9]  STAGE S5 COOLING
     - required_cooling_estimate (heuristic), cooling_adequacy = 0.63 -> risk HIGH
     - warp-risk sub-model scored
     - all five stage vectors now complete

[10] STAGE S6 QUALITY_INSPECTION -> FUSION
     - feature builder assembles: 18 raw + 25 stage + 14 engineered + context
     - vector order read from feature_schema.json of the active model bundle
     - XGBoost predicts raw score -> isotonic calibrator -> p_fail = 0.71
     - threshold 0.42 (tuned, stored in the bundle) -> label FAIL
     - Isolation Forest global anomaly score computed
     - out-of-distribution check: 2 features outside training percentile band
       -> caveats populated

[11] RISK ENGINE
     - model component: p_fail 0.71 -> band HIGH
     - anomaly component: thermal anomaly true -> escalate confidence caveat
     - constraint component: 2 warnings, 0 critical -> no BLOCK
     - operational risk = HIGH, drivers listed
     NOTE: model confidence (how sharp the probability is) and operational risk
     (how bad the situation is) are computed and displayed separately. A model
     can be confidently predicting a mildly risky state, or unconfident about a
     dangerous one. Collapsing them into one number is a design error this
     system explicitly avoids.

[12] EXPLAINABILITY
     - TreeSHAP with the cached background sample -> signed attributions
     - features mapped to owning stages -> stage_risk_mass
       COOLING 58% | THERMAL 24% | PACKING 11% | other 7%
     - persist predictions row with explanation JSON
     - emit prediction.ready, run.completed

[13] FRONTEND renders the result card, the SHAP bars, the stage risk donut,
     and the out-of-distribution caveat banner.

[14] USER clicks "WHAT SHOULD I CHANGE?"  -> POST /api/v1/recommend
     {run_id, locked: ["machine_id","wall_thickness_mm","cavity_count"],
      max_changes: 3}

[15] RECOMMENDATION FUNNEL
     Stage 1  remedy table consulted for COOLING(HIGH), THERMAL(MEDIUM),
              PACKING(MEDIUM) -> 28 directed candidates
     Stage 2  coordinate search, 7 levels per actionable parameter within
              step limits and admissible windows -> 412 candidates evaluated
              (each: partial re-sim from the earliest invalidated stage +
               one fusion inference; nearly all stage results cache-hit)
     Guard    box -> hard constraints -> couplings -> in-distribution ->
              step size -> stability (20 perturbations each for finalists)
              -> 67 candidates survive
     Stage 3  differential evolution seeded from the top 5, 2,000-evaluation
              budget, same guard applied to every proposal
     Ranking  improvement - change_cost - instability - extrapolation
              - cycle_time_penalty
     Output   3 options, each with a constraint stamp and a cycle-time delta
     Persist  recommendations rows
     Latency  ~3.1 s (progress indicator shown from 1 s)

[16] USER sends Option 2 to the What-If workbench.
     POST /api/v1/whatif {base_run_id, overrides:{melt_temp_c: 262,
                          cooling_time_s: 26, hold_pressure_bar: 655}}

[17] WHAT-IF ENGINE
     - dependency graph: melt_temp invalidates from S2; cooling_time from S5;
       hold_pressure from S4  -> earliest invalidated = S2
     - S1 reused from the parent run's cached result
     - S2..S5 re-executed (mostly cache misses; ~9 ms total)
     - fusion re-inference -> p_fail 0.12, label PASS
     - child run persisted with parent_run_id
     - diff assembled: parameters, per-stage health/risk transitions,
       prediction flip, cycle-time impact +5.0 s (+13%)

[18] FRONTEND renders the A/B diff; the stage pipeline shows COOLING moving
     from HIGH (orange) to LOW (green), with the flip badge on the result card.

[19] USER drags the cooling-time slider from 26 s to 24 s.
     Debounced 150 ms -> POST /whatif with the single override
     -> only S5 + fusion recomputed -> ~5 ms -> live update.
     A 1-D sweep (POST /whatif/sensitivity) draws the p_fail curve across the
     cooling-time window with the flip point marked at 22.4 s.

[20] USER saves the scenario. It appears in History under the parent run's
     lineage tree, and in Analytics (excluded from failure-rate aggregates
     because is_whatif = true).

[21] Every step above is logged with the run correlation ID; the prediction,
     its explanation, the recommendations, and the model version are all
```

```
    persisted, making the entire interaction reproducible and auditable.
```

---

29. Closing Statement on Claims

The system, restated one final time in its own terms:

- **It predicts** a binary quality outcome and a calibrated failure probability, from process parameters, using a model fitted to a primarily synthetic corpus built on documented process-engineering relationships.
- **It simulates** a manufacturing workflow at the software level: a state machine advancing a virtual batch through six stages, with rule-based and model-based evaluation at each stage. It does not simulate physics.
- **It recommends** parameter configurations that lower the *model's* predicted failure probability, filtered by hard, documented admissibility constraints and checked for stability and in-distribution validity.
- **It cannot claim**, and must never be presented as claiming, that any recommendation improves real product quality, scrap rate, or process capability. That claim requires physical trial runs, metrology, and statistical process validation on actual equipment — none of which exist within this project's scope.

Holding those four statements apart, consistently, in the code, the interface, the report, and the viva, is what makes the project academically defensible. The engineering depth is what makes it substantial. The interface is what makes it memorable. All three are achievable in twelve weeks by a disciplined team that refuses to build anything on the prohibition list.